

[object Object] [object Object]

누구를 위한 책?

코딩 0회차, 터미널·Git 한 번도 안 써본 분이

마우스 클릭 + 복사만으로 실전 풀스택 웹 시스템을 처음부터 끝까지 만들어 보는 책.

다 끝나면 손에 남는 것

- 인터넷 주소(GitHub Pages)에 올라간 신청 사이트
- 인터넷 엑셀(DB + Storage)
- 신청 들어오면 자동으로 누적되는 엑셀 파일
- 관리자 페이지(목록·상세·다운로드·초기화)
- 보안 잠금(RLS · XSS · 서명 URL)
- Push 한 번이면 자동 배포(GitHub Actions)
- 운영 점검표·인수인계 문서·자동 백업까지

[object Object] [object Object]

- **0장.** 준비물 — 계정 만들고 프로그램 깔기 ★
- **1장.** Cursor로 "Hello ○○아파트" 페이지 만들기 ★★
- **2장.** GitHub에 처음 올리기 ★★
- **3장.** GitHub Pages로 진짜 인터넷 주소 만들기 ★★ 🍷

[object Object]

- **4장.** Supabase 프로젝트 만들고 "인터넷 엑셀" 열기 ★★
- **5장.** 입주민 등록부 테이블 만들기 (SQL 복붙) ★★★

[object Object]

- **6장.** 동·호·이름·휴대폰 입력 칸 만들기 ★★★ 🍷
- **7장.** "다음" 누르면 진짜 입주민인지 확인하기 ★★★
- **8장.** 평형별 옵션 화면 만들기 ★★★
- **9장.** 신청서 작성 폼 (개인정보 입력) ★★★
- **10장.** 손가락·마우스 서명 칸 만들기 ★★★
- **11장.** 신청 데이터 인터넷 엑셀(DB)에 진짜 저장 ★★★★★
- **12장.** 신청 들어올 때마다 엑셀에 자동 누적 ★★★★★

[object Object]

- **13장.** 관리자 로그인 + 신청 목록 화면 ★★★
- **14장.** 관리자 — 신청서 상세 + 엑셀 안전 다운로드 ★★★★★
- **15장.** 관리자 — 시즌 종료용 "초기화" 버튼 ★★★★★

[object Object]

- **16장.** GitHub Actions로 자동 배포 ★★★
- **17장.** 보안 최종 잠금 (RLS · 권한 · XSS) ★★★★★

- **18장.** 운영 점검표 + 백업 ★★

[object Object]

- **19장.** 자주 만나는 에러 30선 — 빠른 해결 카탈로그 ★
- **20장.** 시즌 시작/종료 + 인수인계 가이드 ★★

[object Object]

- **부록 A.** 모든 SQL 한 페이지 (복붙용)
- **부록 B.** 모든 Edge Function 코드 모음
- **부록 C.** 환경변수·시크릿 전체 표
- **부록 D.** 더 안전·확장 (선택 사항)

★ = 클릭만 / ★★ = 약간 복붙 / ★★★ = AI 활용 + 검증 / ★★★★ = SQL/Edge Function / ★★★★★ = 디버깅 동반

☒ = 체크포인트 (지금까지 한 게 잘 됐는지 한번 멈춰서 확인하는 챕터)

[object Object]

이 장에서 완성하는 것

컴퓨터 한 대에 개발 도구 1개 + 인터넷 계정 3개가 준비됩니다.

다 끝나면 본격 작업 들어갈 준비 완료입니다.

소요 시간: 1.5~2시간

난이도: ★ (가장 쉬움. 클릭만 하시면 됩니다)

[object Object]

항목	뭐예요? (1줄 비유)
Cursor	"AI가 코드 대신 짜주는 메모장"
Node.js	"코드를 컴퓨터가 읽게 해주는 번역기"
GitHub 계정	"코드 인터넷 클라우드 (구글 드라이브 비슷)"
Supabase 계정	"인터넷에 있는 엑셀 창고 + 자물쇠"
Gmail 계정	"위 3개 가입할 때 쓰는 이메일 (이미 있으면 패스)"

💡 위 5개 모두 무료입니다. 신용카드 안 넣으셔도 됩니다.

[object Object]

이미 쓰는 메일이 있으면 그거 쓰셔도 됩니다. 새로 만드신다면:

1. <https://accounts.google.com/signup> 접속

2. 이름·아이디·비밀번호 입력 → 다음 계속 누르기

3. 휴대폰 번호 인증 (받은 6자리 숫자 입력)

[스크린샷: Google 가입 첫 화면 — “Google 계정 만들기”]

✔ 이런 화면이 보이면 성공: 오른쪽 위에 동그란 내 프로필 아이콘이 뜬.

[object Object]

1. <https://github.com> 접속 → 오른쪽 위 Sign up 클릭

[스크린샷: GitHub 첫 화면 — 오른쪽 위 Sign up 버튼]

2. 이메일·비밀번호·아이디 입력

○ 아이디는 영어로 짧게 (예: `nerdplanet2026`) — 나중에 사이트 주소에 들어갑니다

3. 캡차(“퍼즐 맞추기”) 풀고 → Create account

4. 가입 후 "What do you do?" 같은 질문 나오면 아무거나 선택 → **Continue**

[스크린샷: GitHub 가입 양식 — Email/Password/Username 입력칸]

✅ 이런 화면이 보이면 성공: 왼쪽 위에 **고양이 옥토켓** 로고, 오른쪽 위에 내 프로필 아이콘이 뜬.

⚠️ 이런 화면이 뜨면:

- "Username already taken" → 아이디가 이미 있음. 뒤에 숫자 붙이세요 (예: `nerdplanet2026`)
- 가입 메일이 안 와요 → 스팸함 확인. 거기도 없으면 5분 후 한 번 더 시도

[object Object]

1. <https://supabase.com> 접속 → 오른쪽 위 **Start your project** 클릭

[스크린샷: Supabase 홈 — Start your project 초록 버튼]

2. **Continue with GitHub** 클릭 (방금 만든 GitHub 계정으로 자동 가입됨)
3. GitHub에서 "Authorize Supabase" 화면 나오면 **Authorize** 클릭

[스크린샷: GitHub Authorize Supabase 권한 요청 화면]

✅ 이런 화면이 보이면 성공: Supabase 대시보드에 **New project** 라는 초록 버튼이 보임.

💡 프로젝트는 아직 만들지 마세요. 4장에서 같이 만듭니다.

[object Object]

Cursor = AI 비서가 옆에 앉아 코드를 대신 짜주는 메모장입니다.

1. <https://cursor.com> 접속 → **Download** 클릭
2. 본인 컴퓨터 종류에 맞는 거 선택
 - **Mac (M1/M2/M3)** → "Download for Mac (Apple Silicon)"
 - **Mac (Intel)** → "Download for Mac (Intel)"
 - **Windows** → "Download for Windows"

[스크린샷: Cursor 다운로드 페이지 — OS 선택 버튼]

3. 다운로드된 파일 더블클릭 → **응용프로그램 폴더로 드래그** (Mac) / **Next 계속** (Windows)
4. Cursor 첫 실행 → 로그인 화면이 뜨면 **GitHub 계정으로 로그인**

[스크린샷: Cursor 첫 실행 화면 — GitHub 로그인 버튼]

5. AI 모델 선택 화면이 뜨면 **무료 옵션** 선택 (`claude-3-haiku` 같은 거 무료)

[스크린샷: Cursor 메인 화면 — 왼쪽 파일 트리 + 가운데 Welcome]

✅ 이런 화면이 보이면 성공: 검은 배경 메모장 같은 화면이 뜨고, 오른쪽에 채팅창이 보임.

[object Object]

1. <https://nodejs.org> 접속
2. 페이지 가운데 두 개 버튼 중 왼쪽 (LTS) 클릭

[스크린샷: Node.js 다운로드 페이지 — LTS 버튼이 강조됨]

3. 다운로드된 파일 더블클릭 → 계속 / Next 만 누름 (옵션 건드리지 말기)

[object Object]

1. Cursor 열기
2. 위 메뉴 Terminal → New Terminal 클릭

[스크린샷: Cursor 상단 Terminal 메뉴 — New Terminal 항목]

3. 아래쪽에 검은 창이 열림. 거기에 아래 한 줄 그대로 붙여넣고 Enter:

```
node --version
```

✅ 이런 게 뜨면 성공: `v22.11.0` 같은 버전 숫자

⚠️ `command not found: node` 가 뜨면 → 설치가 제대로 안 됨. **Cursor** 한 번 꺾다 다시 켜고 같은 명령 다시 입력. 그래도 안 되면 Node.js 다시 설치.

[object Object]

영어 메뉴가 어렵게 느껴지시면:

1. 왼쪽 사이드바 사각형 4개 아이콘 (Extensions) 클릭
2. 검색창에 `Korean` 입력
3. **Korean Language Pack for Visual Studio Code** → Install
4. 오른쪽 아래 "Restart" 뜨면 클릭

[스크린샷: Extensions에서 Korean 검색 결과]

✅ Cursor 메뉴가 한국어로 바뀌어 있음.

[object Object]

아래 6개가 전부 ✅ 면 1장으로 가셔도 됩니다.

- ☐ Gmail로 로그인됨 (또는 기존 메일 사용 결정)
- ☐ **GitHub** 계정 생성 + 로그인됨 → 내 아이디 메모:
- ☐ **Supabase** 대시보드 접속됨 (New project 버튼 보임)
- ☐ **Cursor** 설치 + 실행됨 + 채팅창 보임
- ☐ **Node.js** 설치 → `node --version` 에서 버전 숫자 나옴
- ☐ (선택) Cursor 한국어 메뉴

📌 다음 장 미리보기

1장에서는 Cursor를 열어 빈 **React 프로젝트 1개**를 만들어 봅니다. AI에게 “React 빈 프로젝트 만들어 줘” 한 줄만 던지면 됩니다.

🎯 Cursor에 그대로 복사 (1장에서 쓸 첫 프롬프트 미리보기)

내 컴퓨터의 바탕화면에 yyc-options 라는 폴더를 만들고,
그 안에 Vite + React 빈 프로젝트를 초기화해 줘.
진행하기 전에 어떤 명령어를 실행할지 먼저 보여줘.

💪 여기까지 오신 분께

0장은 “계정 만들기”라 지루했을 수 있는데, 이게 끝나면 **이제부터는 진짜 만드는 재미**입니다. 한숨 돌리시고 1장으로 가세요.

I'm planning out the structure for a beginner-friendly Chapter 0 guide that walks users through setup step-by-step, with visual screenshots, clear click-and-copy instructions, success verification checks, and a callout section for common mistakes.

[object Object]

이 장에서 완성하는 것

Cursor에서 AI에게 “빈 사이트 만들어 줘” 한 줄만 시키면, 내 컴퓨터에서 사이트가 떠 있고 거기에 “Hello ○○아파트”라고 쓰여 있게 됩니다.

소요 시간: 약 1시간

난이도: ★★ (Cursor에 처음 명령 던져 보는 단계)

[object Object]

용어	1줄 비유
React	"버튼·입력칸 같은 화면 부품을 조립하는 레고"
Vite	"그 레고를 빠르게 조립해서 화면에 띄워주는 기계"
터미널	"Cursor 아래쪽 검은 창. AI가 명령을 실행하는 곳"
localhost	"내 컴퓨터에서만 잠깐 보이는 사이트 주소"

 위 단어 외울 필요 없습니다. **클릭만 하시면 됩니다.**

[object Object]

1. **Cursor 실행** (0장에서 깐 거)

2. 위 메뉴 **File → New Window**

[스크린샷: Cursor File 메뉴 — New Window 항목]

3. 새 창이 뜨면 가운데 큰 “Open Folder” 버튼이 보입니다.
아직 누르지 마세요. 폴더는 AI가 만들어 줄 거예요.

 이런 화면이 보이면 성공: 가운데 “Get Started” 또는 빈 환영 화면.

[object Object]

1. 위 메뉴 **Terminal → New Terminal**

2. 화면 아래쪽에 검은 창이 열립니다.

[스크린샷: Cursor 화면 아래쪽에 검은 터미널 창이 열린 모습]

3. 검은 창 위에 보이는 경로(예: ) 가 어디든 상관 없습니다.

 이런 게 보이면 성공: 깜빡이는 커서가 있는 검은 입력창.

[object Object]

검은 창에 아래 한 줄을 그대로 복사 → 붙여넣고 → Enter:

```
cd ~/Desktop
```

✅ 입력창 앞부분이 `.../Desktop %` 처럼 바뀜.

⚠️ `No such file or directory` 가 뜨면 → 한국어 윈도우는 `~/Desktop` 대신 `~/바탕\ 화면` 일 수 있어요.
Cursor 채팅에 “바탕화면으로 이동하는 명령 알려줘” 라고 물어 보면 됩니다.

[object Object]

오른쪽에 있는 **Cursor 채팅창** 클릭. 만약 안 보이면 위 메뉴 **View → Open Chat** 또는 단축키 `Cmd + L` (맥) / `Ctrl + L` (윈도).

[스크린샷: Cursor 오른쪽 채팅창 — 입력창 클릭한 모습]

🎯 Cursor에 그대로 복사

지금 내 위치(바탕화면)에 `yyc-options` 라는 폴더를 만들고,
그 안에 `Vite + React (JavaScript)` 빈 프로젝트를 초기화해 줘.

진행 단계:

- 1) 어떤 명령어를 실행할지 먼저 보여줘
- 2) 내가 "go" 라고 답하면 그때 실행해 줘
- 3) 실행 끝나면 "다음에 할 일" 한 줄로 알려줘

엔터 치면 AI가 명령어를 보여 줍니다 (대략 `npm create vite@latest yyc-options ...` 같은 줄).

[스크린샷: Cursor 채팅창에 AI 답변 — 명령어 미리보기]

내용을 숙 본 다음 채팅창에 한 글자만:

```
go
```

✅ AI가 터미널에 명령을 자동 입력하기 시작합니다.

[object Object]

명령 실행 도중 Cursor 가운데에 “**Run command?**” 같은 노란 박스가 뜹니다.

[스크린샷: Cursor — Run command 권한 요청 박스, Allow / Deny 버튼]

→ **Allow** 클릭 (한 번에 안 끝나면 몇 번 더 나옴 — 모두 Allow)

설치 도중 터미널이 멈춘 듯 보여도 **2~3분 기다리면** 자동으로 끝납니다. 절대 닫지 마세요.

✅ 이런 게 마지막에 보이면 성공:


```
Done. Now run:
  cd yyc-options
  npm install
  npm run dev
```

[object Object]

AI가 위에서 멈췄으면 채팅창에 한 줄 더:

Cursor에 그대로 복사

```
이어서 yyc-options 폴더로 들어가서 npm install 하고,
끝나면 npm run dev 까지 실행해 줘. 권한 박스 나오면 알려줘.
```

Allow 또 한두 번 누르면, 아래쪽 검은 창에 이런 글자가 뜹니다:

```
VITE v8.x.x  ready in 432 ms
→ Local:   http://localhost:5173/
```

[스크린샷: 터미널에 "Local: <http://localhost:5173/>" 출력된 모습]

✅ **Local: <http://localhost:5173/>** 이 보이면 사이트가 떠 있는 상태예요.

[object Object]

1. 위 주소를 **Cmd** 누른 채 클릭 (맥) / **Ctrl +** 클릭 (윈도)
→ 브라우저가 자동으로 열림
2. 또는 그냥 브라우저에 **<http://localhost:5173/>** 직접 입력

[스크린샷: 브라우저 — Vite + React 기본 화면 (회전하는 React 로고)]

✅ React 로고가 빙글빙글 도는 화면이 뜬 = **첫 사이트 띄우기 성공**

⚠️ **흰 화면만 뜨면:** 5초 기다리시고 새로고침. 그래도면 터미널 빨간 글씨 한 줄 복사해 Cursor 채팅에 붙이고 "고쳐 줘".

[object Object]

이제 화면 글자만 바꿔 봅시다.

1. 왼쪽 파일 트리에서 **yyc-options** 폴더 → **src** → **App.jsx** 더블클릭

[스크린샷: 왼쪽 파일 트리에서 src/App.jsx 클릭한 모습]

2. 코드가 화면에 뜨면 그냥 보지 마시고, **AI에게 시켜요.**

Cursor에 그대로 복사

@App.jsx 파일 내용을 전부 지우고,
화면 한가운데에 큰 글씨로 "Hello ○○아파트" 만 보이게 바꿔 줘.
글씨는 진하게, 가운데 정렬.

@App.jsx 라고 입력하면 자동완성에서 그 파일이 떠오. 선택 후 엔터.

3. AI가 코드 변경 안을 보여 줍니다. 위쪽 **Apply / Accept** 버튼 클릭.

[스크린샷: Cursor 채팅 — Apply 버튼이 있는 코드 변경 미리보기]

4. 브라우저로 가서 **새로고침** (**Cmd+R** / **Ctrl+R**).

[스크린샷: 브라우저 — "Hello ○○아파트" 가 가운데 큰 글씨로 보이는 화면]

✅ "Hello ○○아파트" 만 뜨면 성공.

[object Object]

- 사이트 그만 띄우고 싶을 때: 터미널에서 **Ctrl + C** → 다시 띄우려면 **npm run dev**
- Cursor에서 한 거 **자동 저장**됩니다. 따로 저장 안 누르셔도 돼요.

[object Object]

- ☐ 바탕화면에 **yyc-options** 폴더가 생겼다
- ☐ 그 안에 **src**, **package.json** 같은 파일들이 보인다
- ☐ **npm run dev** 했을 때 터미널에 **Local: http://localhost:5173/** 가 떴다
- ☐ 브라우저로 그 주소 들어가면 **"Hello ○○아파트"** 가 보인다
- ☐ 같은 화면이 사라지지 않고 그대로 떠 있다 (터미널 닫지 않기)

[object Object]

화면	원인	해결
command not found: npm	Node.js 설치 실패	0장 다시. Cursor 꺾다 켜기
EADDRINUSE: address already in use 5173	사이트가 이미 떠 있음	다른 터미널에서 Ctrl+C 로 끄기
흰 화면 / Cannot GET /	dev 서버 아직 시작 중	5초 후 새로고침
한국 글자 깨짐	인코딩 문제	Cursor 재시작 한 번
Allow 박스 못 봄	권한 박스 닫혀버림	채팅에 "명령 다시 실행해줘"

📌 다음 장 미리보기

2장에서는 지금 만든 `yyc-options` 폴더를 **GitHub**에 올립니다. Cursor 안에서 클릭 몇 번이면 끝나요. 이게 되면 “내 코드가 인터넷에 백업됨” 상태가 됩니다.

💪 여기까지 오신 분께

코드 한 줄 안 짰는데 사이트 한 페이지를 띄웠어요. **이게 진짜 시작점**입니다. 한 잔 마시고 2장 가시죠.

I'm planning out Chapter 1, which will walk through creating a "Hello OO아파트" page using Cursor, starting with opening the tool and setting up a folder structure, with an estimated completion time of one hour.

Now I need to structure the step-by-step instructions covering everything from launching Cursor through running the dev server and modifying the App component, including screenshots, success checkpoints, and troubleshooting tips, all written in a friendly tone with Cursor prompts highlighted in callout boxes.

```
[object Object]
```

이 장에서 완성하는 것

1장에서 만든 `yyc-options` 폴더가 **GitHub** 인터넷 클라우드에 올라갑니다.

이제 노트북이 부서져도 코드가 안 사라집니다.

소요 시간: 약 1시간

난이도: ★★ (Cursor의 클릭 몇 번 + 짧은 복붙)

[object Object]

용어	1줄 비유
Git	"내 폴더 변경 내역을 자동으로 적어두는 일기장"
GitHub	"그 일기장을 인터넷 클라우드에 올려두는 곳"
Repository (레포)	"내 프로젝트 폴더 1개 = 인터넷에 올라간 1개 칸"
Commit	"지금까지 바꾼 거 한 묶음으로 도장 찍기"
Push	"내 컴퓨터 → GitHub로 업로드"

💡 외울 필요 없습니다. 다 버튼 한 번으로 끝납니다.

```
[object Object] [object Object]
```

<https://github.com> 로그인 → 오른쪽 위 + 버튼 → New repository

[스크린샷: GitHub 오른쪽 위 + 메뉴 — New repository 항목]

```
[object Object]
```

항목	입력값
Repository name	<code>yyc-options</code> (1장 폴더명과 똑같이)
Description	<code>옵선 신청 시스템</code> (아무거나 써도 됨, 비워도 됨)
Public / Private	Private 추천 (나만 보기)
Add a README / .gitignore / license	모두 체크 해제 ⚠️ 중요

[스크린샷: New repository 페이지 — 이름.Private 선택.체크박스 모두 해제 모습]

```
[object Object]
```

✔ 이런 화면이 보이면 성공: "Quick setup" 안내 페이지 (까만 코드 박스 여러 줄)

[스크린샷: GitHub 빈 레포 Quick setup 페이지 — "...or push an existing repository" 박스 표시]

💡 위 페이지에 보이는 까만 코드는 읽지 마세요. Cursor가 알아서 합니다.

[object Object] [object Object]

1장에서 띄워둔 `yyc-options` 창이면 그대로. 닫았으면:

- Cursor → **File** → **Open Folder** → 바탕화면의 `yyc-options` 선택

[object Object]

[스크린샷: Cursor 왼쪽 사이드바 — Git 아이콘(분기) 빨간 동그라미]

[object Object]

[스크린샷: Source Control 패널 — Initialize Repository 버튼]

✅ 왼쪽 패널에 "Changes" 라는 글자 + 파일 목록이 한 무더기 뜬.

[object Object] [object Object]

첫 커밋

[스크린샷: Source Control — Message 입력칸과 Commit 버튼]

[object Object]

"No staged changes..." 라는 팝업이 뜨면 → **Yes** 클릭 (전부 한 번에 묶어준다는 뜻)

✅ 파일 목록이 비고, 아래쪽에 "Publish Branch" 같은 파란 버튼이 새로 뜬.

⚠️ "user.email is not set" 같은 빨간 글씨가 뜨면:

→ 채팅창(`Cmd + L`)에 그대로 던지세요.

🎯 **Cursor에 그대로 복사**

```
Git 첫 commit 하려는데 "user.email is not set" 에러가 떠.  
내 GitHub 가입 이메일과 아이디로 git config 설정하는 명령 알려줘.  
그 후에 다시 commit 해줘.
```

[object Object] [object Object]

[스크린샷: Source Control — Publish Branch 버튼]

[object Object]

브라우저로 이동되면서 “**Authorize Cursor**” 같은 페이지가 뜹니다.

[스크린샷: GitHub Authorize Cursor 화면 — Authorize 초록 버튼]

→ **Authorize** 클릭. Cursor로 자동 돌아옵니다.

[object Object]

→ **Private** 선택 (이미 GitHub에 만든 레포가 Private면 자동 매칭됨)

[object Object]

✅ 사이드바 **Changes** 항목이 비어 있고, 아무 에러도 없음.

[object Object] [object Object]

주소: <https://github.com/내아이디/yye-options>

[object Object]

[스크린샷: GitHub 레포 페이지 — `src`, `package.json`, `index.html` 등 파일 목록이 보임]

✅ 이런 게 보이면 성공: `src/`, `public/`, `package.json`, `index.html` 등 파일이 쭉 나열됨.

⚠️ 여전히 “**Quick setup**” 페이지면: Push가 안 된 상태.

→ Cursor 채팅에:

🎯 Cursor에 그대로 복사

```
지금 main 브랜치 GitHub에 push가 안 된 상태야.  
origin 추가하고 푸시까지 한 번에 해줘.  
내 레포 주소: https://github.com/내아이디/yye-options
```

(`내아이디` 부분만 본인 아이디로 바꿔서 붙여넣기)

[object Object]

GitHub 레포에 `.env` 나 `node_modules` 폴더가 보이면 안 됩니다.

[object Object]

[스크린샷: GitHub 레포 — `node_modules` 가 보이지 않음을 확인하는 모습]

✅ 안 보이면 정상 (Vite가 자동으로 `.gitignore`에 넣어 두었음).

⚠️ `node_modules` 폴더가 보이면:

Cursor에 그대로 복사

내 .gitignore 에 node_modules, .env, .env.local, dist 가 있는지 확인하고
빠진 거 있으면 추가해 줘. 그 후에 변경사항 commit + push 까지 해줘.

[object Object]

매번 코드 고친 다음:

1. 사이드바 나뭇가지 아이콘 클릭
2. Message에 한 줄 (예: 옵션 카탈로그 추가) → ✓ Commit
3. 아래쪽 Sync Changes 또는 ... → Push

[스크린샷: 평소 코드 수정 후 Source Control에 Changes 가 뜬 모습]

💡 막히면 그냥 **Cursor 채팅**에 한 줄:
"지금까지 바꾼 거 commit 메시지 알아서 정해서 push 해줘."

[object Object]

- ☐ GitHub에 **yyc-options** 레포가 만들어져 있다 (Private)
- ☐ Cursor 사이드바에 나뭇가지 아이콘 + 체크된 상태 가 보인다
- ☐ 브라우저에서 <https://github.com/내아이디/yyc-options> 들어가면 **src/**, **package.json** 등 파일이 보인다
- ☐ **node_modules** 폴더가 GitHub에 안 보인다
- ☐ 시험 삼아 **App.jsx** 한 글자 바꾸고 commit + push 했더니 GitHub에서 그 변경이 보인다

[object Object]

화면	원인	해결
Permission denied (publickey)	GitHub 인증 안 됨	Cursor 재시작 후 다시 Push (재인증 팝업이 나옴)
failed to push some refs	원격에 다른 게 있음	채팅창에 "git pull rebase 하고 다시 push 해줘"
nothing to commit	바뀐 게 없음	정상. 그냥 다음 단계로
Sync Changes 누르니 빨간 알림	인증 만료 / 충돌	메시지 그대로 복사해 채팅에 붙이기
node_modules 가 GitHub에 보임	.gitignore 누락	위 2-7 박스 그대로

다음 장 미리보기

3장에서는 GitHub에 올린 이 코드를 **GitHub Pages**로 인터넷 주소까지 만들어 봅니다.
끝나면 친구한테 “이 주소 들어가 봐” 라고 보내줄 수 있는 진짜 사이트가 됩니다.

여기까지 오신 분께

1장에서 “내 컴퓨터에서만 보이던 사이트”의 **소스 코드가 인터넷에 백업**됐어요.
다음 장에서 그 소스를 진짜 인터넷 주소로 띄울 거예요. 잠깐 쉬셨다 가시죠.

이 장에서 완성하는 것

1.2장에서 만든 사이트가 **인터넷 주소**로 떠오릅니다.

친구한테 카톡으로 주소 보내면 폰에서도 보입니다.

주소 모양: `https://내아이디.github.io/yyc-options/`

소요 시간: 약 1시간

난이도: ★★ (복불 위주, 마지막에 5분 기다리는 게 길게 느껴짐)

🚩 1차 체크포인트 챕터입니다.

용어	1줄 비유
GitHub Pages	"GitHub 레포 → 무료 인터넷 사이트"
GitHub Actions	"Push 할 때마다 자동으로 빌드·배포해주는 알바"
워크플로(workflow)	"그 알바한테 시킬 일을 적어둔 메모지"
base path	"내 사이트 주소 끝에 붙는 폴더 이름 (<code>/yyc-options/</code>)"

<https://github.com/내아이디/yye-options>

[스크린샷: GitHub 레포 위 메뉴 — Settings 탭이 빨간 동그라미]

[스크린샷: Settings 왼쪽 메뉴 — Pages 항목]

→ 드롭다운 클릭 → **GitHub Actions** 선택
 ("Deploy from a branch" 가 아니라 **GitHub Actions** 입니다! 🚩)

[스크린샷: GitHub Pages — Source 드롭다운에서 GitHub Actions 선택]

✅ 따로 저장 버튼 없이 자동 저장됨.

⚠ 드롭다운에 "GitHub Actions" 가 안 보이면:

→ 페이지 새로고침 한 번. 그래도 없으면 레포가 **Public**으로 되어 있는지 확인 (무료 계정은 Private에서 Pages 못 쓰는 경우 있음 → Settings → General → 아래쪽 Danger Zone 에서 Public으로 바꾸기).

[object Object]

지금 그대로 배포하면 사이트가 흰 화면만 나옵니다.

이유: GitHub Pages는 주소 끝에 `/yyc-options/` 가 붙는데, Vite가 그걸 모름.

→ AI에게 시키면 끝.

Cursor 채팅창(`Cmd + L` 또는 `Ctrl + L`):

Cursor에 그대로 복사

```
이 프로젝트를 GitHub Pages 에 배포할 거야.  
@vite.config.js 를 수정해서, 환경변수 VITE_BASE 가 있으면 그 값을 base 로,  
없으면 "/" 를 base 로 쓰도록 바꿔줘.  
변경 후 Apply 해줘.
```

[스크린샷: Cursor — vite.config.js 변경안 미리보기 + Apply 버튼]

→ **Apply** 클릭.

 `vite.config.js` 안에 `base: process.env.VITE_BASE || "/"` 같은 줄이 들어가 있으면 OK.

[object Object]

이게 핵심. 한 번 만들면 앞으로 **Push만 하면 자동으로 사이트 갱신**됩니다.

[object Object]

Cursor에 그대로 복사

```
.github/workflows/pages.yml 파일을 새로 만들어 줘.  
내용:  
- main 브랜치에 push 되면 자동 실행  
- Node 22 사용  
- npm ci → npm run build (env: VITE_BASE = /yyc-options/)  
- dist 폴더를 GitHub Pages 로 배포  
표준 actions/configure-pages, actions/upload-pages-artifact, actions/deploy-pages 를 써줘.  
만든 다음 Apply 해줘.
```

[스크린샷: Cursor — pages.yml 내용 미리보기]

→ **Apply** 클릭.

[object Object]

왼쪽 파일 트리에서 `.github/workflows/pages.yml` 가 새로 보여야 함.

[스크린샷: 왼쪽 파일 트리에서 .github/workflows/pages.yml 항목]

✅ 파일 안에 `name: Deploy GitHub Pages`, `branches: [main]`, `VITE_BASE: /yyc-options/` 같은 줄이 보이면 OK.

⚠️ `yyc-options` 가 아닌 다른 이름으로 만들었다면 → 위 워크플로 안의 `VITE_BASE` 값도 그 이름에 맞게 바꿔야 합니다. AI에게:

🌀 Cursor에 그대로 복사

@pages.yml 의 VITE_BASE 를 /내실제레포이름/ 으로 바꿔줘.

[object Object]

2장에서 배운 그대로:

1. 사이드바 나뭇가지 아이콘
2. Message: `GitHub Pages 자동 배포 추가`
3. ✓ **Commit** → **Sync / Push**

[스크린샷: Source Control — 2개 변경(.github/workflows/pages.yml, vite.config.js) commit 모습]

또는 막히면:

🌀 Cursor에 그대로 복사

지금 바뀐 거 commit 메시지 정해서 push 해줘.

[object Object] [object Object]

[스크린샷: GitHub 레포 위 메뉴 — Actions 탭]

[object Object] [object Object]

[스크린샷: Actions — Deploy GitHub Pages 노란 점(in progress) 모습]

[object Object]

[스크린샷: Actions — Deploy GitHub Pages 초록 ✅ 모습]

⚠️ 빨간 ❌ 가 뜨면: 그 줄 클릭 → 빨간 단계 클릭 → 빨간 글씨 한 줄 복사 → Cursor 채팅에:

Cursor에 그대로 복사

GitHub Actions 빨간 에러 떴어. 메시지 붙여넣을게. 원인이랑 고치는 법 알려주고
필요하면 코드/yml 수정해서 Apply 해줘.

[여기에 빨간 글씨 한 줄 붙여넣기]

[object Object] [object Object]

[스크린샷: 워크플로 상세 — 위쪽에 연한 파란 글씨로 <https://내아이디.github.io/yyc-options/> 표시]

또는

[object Object]

[스크린샷: Settings → Pages — Your site is live at <https://...> 안내]

[object Object]

[스크린샷: 브라우저 — “Hello OO아파트” 가 인터넷 주소에 떠 있는 모습]

✅ “Hello OO아파트” 가 인터넷 주소에서 보이면 = 진짜 사이트가 살아 있는 상태.

⚠️ 404 가 뜨면: 5분만 더 기다리세요. GitHub Pages가 처음 활성화될 때 1~10분 지연 정상.

⚠️ 흰 화면만 뜨면: `vite.config.js` 의 `base` 가 `/yyc-options/` 가 맞는지 확인 (3-3 단계).

[object Object] [object Object] [object Object]

[스크린샷: 휴대폰 화면 — 같은 페이지가 폰에서 보이는 모습]

✅ 친구 폰에서 보이면 3장 합격 + 1차 체크포인트 통과.

[object Object]

코드 수정 → Cursor에서 commit + push

↓

GitHub Actions 가 자동으로 빌드

↓

30초~1분 후 인터넷 주소가 자동으로 갱신

= 한 번 세팅하면 끝. 앞으로 push만 하면 됩니다.

[object Object]

- ☐ Settings → Pages Source 가 GitHub Actions 로 되어 있다
- ☐ `.github/workflows/pages.yml` 파일이 GitHub 레포에서도 보인다

- ☐ **Actions** 탭에 **Deploy GitHub Pages** 워크플로가  초록으로 끝났다
- ☐ `https://내아이디.github.io/yc-options/` 들어가면 **"Hello OO아파트"** 가 보인다
- ☐ 다른 사람 폰에서도 같은 페이지가 뜬다
- ☐ 시험 삼아 `App.jsx` 글자 한 글자 바꿔 push → 1분 후 인터넷 주소도 같이 바뀌었다

[object Object]

화면	원인	해결
<code>404 - File not found</code>	Pages 활성화 직후 지연	5~10분 기다리기
흰 화면만 뜸	<code>base</code> 설정이 <code>/</code> 임	<code>vite.config.js</code> 의 <code>base: "/yc-options/"</code> 확인
Actions 가 안 돌아감	Source 가 "Branch" 로 되어 있음	Settings → Pages → GitHub Actions 로 다시
<code>Pages site failed</code> 빨간 에러	권한 부족	Settings → Actions → General → Workflow permissions → Read and write 체크
폰에선 글자 잘림	CSS 미적용	정상. 6장에서 모바일 스타일 잡습니다
폰에서 안 들어가짐	같은 와이파이라 캐시 꼬임	폰을 LTE로 바꿔 다시

🔪 다음 장 미리보기

4장부터 본격 데이터 다루기. **Supabase 프로젝트**를 만들고 "인터넷 엑셀" 한 칸을 열어봅니다.
이게 끝나면 "저장된 신청자 데이터" 를 다룰 수 있는 토대가 생깁니다.

👉 여기까지 오신 분께

코드 0회차에서 이미 인터넷에 본인 사이트가 떠 있는 사람이 됐어요.
명함에 본인 사이트 주소 한 줄 적어도 됩니다 😊
한 잔 마시고 4장 가시죠.

The user confirmed they want to continue, so I'm moving on to chapter 3 about creating a real internet address using GitHub Pages. I'll structure it the same way as the previous chapters, showing the final result, time estimate, and starting the checklist.

The first checkpoint is getting the page live so a friend can view it on their phone and see the "Hello" message. I need to cover enabling GitHub Pages in settings, configuring the Vite base path, setting up the GitHub Actions workflow file, pushing the changes, monitoring the build process, confirming the Pages URL, and collecting common errors that might come up.

[object Object]

이 장에서 완성하는 것

Supabase 가입 → 프로젝트 1개 생성 → 빈 테이블 1개 만들기.

5장에서 진짜 입주민 데이터를 넣을 빈 박스를 미리 준비합니다.

소요 시간: 약 1시간

난이도: ★★ (대부분 클릭, SQL은 5장에서)

[object Object]

용어	1줄 비유
Supabase 프로젝트	"회사 1개 = 인터넷 엑셀 참고 1등"
Database	"그 참고 안의 엑셀 파일 (여러 시트=테이블)"
Table (테이블)	"엑셀 시트 1장"
Row / Column	"엑셀의 행 / 열"
API URL · API Key	"내 사이트가 이 참고에 들어갈 때 쓰는 주소·열쇠"

[object Object] [object Object]

<https://supabase.com/dashboard> → 0장에서 만든 GitHub 계정으로 로그인.

[object Object]

[스크린샷: Supabase 대시보드 — New project 초록 버튼]

[object Object]

항목	입력값
Organization	자동 생성된 거 그대로
Project name	yyc-options
Database Password	무작위 generate 클릭 → 메모장에 따로 저장 ⚠
Region	Northeast Asia (Seoul)
Pricing plan	Free

[스크린샷: New project 폼 — 이름·비번 generate·Seoul 지역 선택]

[object Object]

→ 1~2분 대기 (회사 설립 중)

✅ 이런 화면이 보이면 성공: 왼쪽 사이드바에 🏠 Home, 🗄 Database, 🗝 Authentication 같은 메뉴가 보임.

⚠ 비밀번호 잃어버리면: Settings → Database → Reset password 가능. 그래도 메모장에 적어 두는 게 편함.

[object Object]

이건 5장 이후 코드에서 계속 씁니다. 메모장에 미리 따로 적어 두세요.

[object Object] [object Object]

[스크린샷: Project Settings → API 탭]

[object Object]

이름	어디에 있나	형태
Project URL	위쪽 "Project URL" 박스	https://abcd1234.supabase.co
anon (public) key	"Project API Keys" 의 첫 번째 줄	eyJhbGc... (긴 문자열)

[스크린샷: API 페이지 — Project URL과 anon key의 Copy 버튼]

✅ 메모장에 두 줄 다 들어 있음.

⚠ service_role 라고 쓰여 있는 두 번째 키는 절대 복사·공유하지 마세요. 17장에서 따로 다룹니다.

[object Object]

진짜 테이블은 5장에서 SQL로 만들 거예요. 여기서선 "테이블이 뭔지" 클릭으로 한 번 체험.

[object Object]

[스크린샷: Supabase 왼쪽 메뉴 — Table Editor]

[object Object] [object Object]

항목	값
Name	test_table
Description	(비워두기)
Enable Row Level Security	체크 OFF (연습용이라)

→ Save 클릭

[스크린샷: New table 모달 — name과 RLS 체크박스]

[object Object]

테이블 화면 위쪽 + Insert → Insert row → id 자동 → Save

[스크린샷: Insert row — id만 있는 한 줄 추가]

✅ 표에 한 줄이 보임 = 인터넷 엑셀에 데이터 한 줄 넣은 상태.

[object Object]

위 메뉴 ... → Delete table → DELETE 입력 → 확인
(일부러 비우는 거라 다음 장에 영향 없음)

[object Object]

- ☐ Supabase에 yyc-options 프로젝트가 떠 있다
- ☐ 메모장에 Project URL + anon key 두 줄이 적혀 있다
- ☐ 데이터베이스 비밀번호도 메모장에 적혀 있다
- ☐ Table Editor 들어가면 빈 화면이 뜬다
- ☐ (연습) 테이블 만들고 → 줄 넣고 → 지우는 흐름을 한 번 해봤다

[object Object]

화면	원인	해결
프로젝트 만들고 30분 지나도 표시 안 됨	프리 플랜 가끔 지연	F5. 그래도면 다시 만들기
API 페이지에 키가 회색	마우스 올리면 "Reveal" 버튼 보임	Reveal 후 복사
Region 못 고름	일부 국가에서 잠금	가까운 다른 region (Tokyo 등) 선택
비번 강도 약함 경고	너무 짧음	generate 버튼 한 번 더

📌 다음 장 미리보기

5장에선 진짜 입주민 등록부 테이블을 SQL로 만듭니다. SQL 박스에 그대로 복사 한 번이면 끝.

[object Object]

이 장에서 완성하는 것

"등록부" 테이블 + 진짜/가짜 입주민 5명 + 검증 함수까지 한 번에.

7장에서 "이 사람 진짜야?" 검증할 때 이 데이터가 쓰입니다.

소요 시간: 약 1.5시간

난이도: ★★★ (SQL 첫 등장 — 그래도 복붙)

[object Object]

용어	1줄 비유
SQL	"엑셀에 'A1 셀에 이거 넣어' 라고 적어주는 메모"
CREATE TABLE	"새 시트 만들기"
INSERT INTO	"새 행 추가"
RLS (Row Level Security)	"이 시트는 비밀번호 가진 사람만 봐"
RPC (함수)	"엑셀 매크로 — 이름만 부르면 안에서 알아서 처리"
SECURITY DEFINER	"이 매크로는 만든 사람 권한으로 돈다 (RLS 무시)"

[object Object] [object Object]

[스크린샷: 왼쪽 메뉴 — SQL Editor 항목]

[object Object]

[스크린샷: SQL Editor — 새 쿼리 작성 화면]

✅ 가운데에 큰 검은 입력창이 떠 있음.

[object Object]

아래 SQL 전체를 그대로 복사 → 가운데 큰 칸에 붙여넣기 → 오른쪽 아래 **Run** (또는 **Cmd + Enter**)

```

-- 1) 등록부 테이블
CREATE TABLE IF NOT EXISTS public.yyc_resident_registry (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  dong text NOT NULL,
  ho text NOT NULL,
  contractor_name text NOT NULL,
  phone_tail text NOT NULL,
  type_key text NOT NULL,
  created_at timestampz NOT NULL DEFAULT now(),
  CONSTRAINT yyc_resident_phone_chk CHECK (phone_tail ~ '^[0-9]{4}$'),
  CONSTRAINT yyc_resident_unique UNIQUE (dong, ho, contractor_name, phone_tail)
);

-- 2) RLS 켜고, 직접 SELECT 금지 (RPC 통해서만 조회)
ALTER TABLE public.yyc_resident_registry ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON public.yyc_resident_registry FROM PUBLIC;

-- 3) 검증 함수 (이름·동호·휴대폰뒤4 다 맞으면 type_key 한 줄 반환)
DROP FUNCTION IF EXISTS public.verify_yyc_resident(text, text, text, text);

CREATE OR REPLACE FUNCTION public.verify_yyc_resident(
  p_dong text,
  p_ho text,
  p_contractor text,
  p_phone_tail text
)
RETURNS TABLE(type_key text)
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path = public
AS $$
  SELECT r.type_key
  FROM public.yyc_resident_registry r
  WHERE r.dong = regexp_replace(coalesce(p_dong,''), '\D', '', 'g')
    AND r.ho = regexp_replace(coalesce(p_ho,''), '\D', '', 'g')
    AND r.contractor_name = trim(regexp_replace(coalesce(p_contractor,''), '\s+', ' ', 'g'))
    AND r.phone_tail = regexp_replace(coalesce(p_phone_tail,''), '\D', '', 'g')
  LIMIT 1;
$$;

REVOKE ALL ON FUNCTION public.verify_yyc_resident(text, text, text, text) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.verify_yyc_resident(text, text, text, text) TO anon, authentic

```

[스크린샷: SQL Editor — 위 코드 붙여넣고 오른쪽 Run 버튼]

✅ 아래 결과창에 **"Success. No rows returned"** 가 뜨면 성공 (테이블·함수가 만들어졌다는 뜻).

⚠️ 빨간 에러가 뜨면: 메시지 한 줄 그대로 복사 → Cursor 채팅에:

Cursor에 그대로 복사

Supabase SQL Editor 에서 이 SQL 돌렸는데 에러 났어. 원인이랑 고친 SQL 줘.
[에러 메시지 붙여넣기]

[object Object]

새 쿼리 창에 또 복붙 → Run.

```
INSERT INTO public.yyc_resident_registry
(dong, ho, contractor_name, phone_tail, type_key)
VALUES
('101', '1504', '임동우', '5604', '59A'),
('101', '1101', '홍길동', '0001', '52A'),
('102', '203', '이수민', '1234', '48B'),
('102', '901', '박지훈', '5678', '65A'),
('103', '1801', '정수정', '9999', '79')
ON CONFLICT DO NOTHING;
```

[스크린샷: SQL Editor — INSERT 5줄 + Success]

✅ "Success. 5 rows" 비슷한 메시지.

[object Object]

왼쪽 **Table Editor** → **yyc_resident_registry** 클릭

[스크린샷: Table Editor — 입주민 5줄 표시]

✅ 표에 5줄 보이면 OK.

💡 본인 이름·동호로 1줄 추가해 두면 7장 테스트할 때 편함. 위 INSERT 형식 그대로 본인 정보 1줄 더 넣고 Run.

[object Object]

```
SELECT * FROM public.verify_yyc_resident('101', '1504', '임동우', '5604');
```

✅ **type_key** 칸에 **59A** 한 줄이 떠야 함.

SELECT * FROM ... ('101', '1504', '임동우', '9999') 처럼 휴대폰 뒷자리 다르면 → **0 rows** (정상).

[스크린샷: SQL Editor — 검증 함수 호출 결과 1줄]

[object Object]

- ☐ **Table Editor** → **yyc_resident_registry** 가 보인다

- ☐ 그 안에 입주민 **5줄 이상** 들어 있다
- ☐ 본인 정보로 한 줄 더 넣어 두었다 (선택)
- ☐ `SELECT * FROM verify_yyc_resident(...)` 호출 시 본인 평형이 나온다
- ☐ 일부러 틀린 값으로 호출하면 **0 rows** 나온다

[object Object]

화면	원인	해결
relation "yyc_resident_registry" already exists	이미 만든 거 포함	정상. 무시
permission denied	보호 모드	SQL Editor 위쪽 "Read only mode" 끄기
INSERT 했는데 0 rows	UNIQUE 충돌	같은 동·호·이름·뒷4가 이미 있음. 정상
함수 호출에 매번 0 rows	입력값 공백/한자	한글 이름 정확히, 휴대폰 4자리 숫자만

📌 다음 장 미리보기

6장에서 사이트 첫 화면에 **입력칸 4개 + 다음 버튼**을 만들어 봅니다. AI 시키면 끝.

```
[object Object]
```

이 장에서 완성하는 것

사이트 첫 화면이 "Hello" 가 아니라

동 / 호 / 계약자명 / 휴대폰 뒷4자리 입력칸 4개 + "다음" 버튼으로 바뀝니다.

빈 칸이면 다음 버튼이 비활성화 됩니다.

소요 시간: 약 1.5시간

난이도: ★★★ (Cursor가 거의 다 해줌)

🏁 2차 체크포인트 챕터입니다.

```
[object Object]
```

용어	1줄 비유
컴포넌트	"한 화면 부품 (입력칸·버튼·박스 등)"
State (상태)	"사용자가 입력한 값을 잠깐 기억해 두는 메모지"
disabled	"비어 있을 때 버튼 회색으로 잠그기"

```
[object Object]
```

Cursor 터미널(아래 검은 창)에서:

```
npm run dev
```

브라우저 → <http://localhost:5173> (또는 5174...)

지금은 "Hello OO아파트" 가 보이는 상태.

```
[object Object]
```

오른쪽 채팅창 (Cmd + L):

Cursor에 그대로 복사

@App.jsx 의 화면을 다음과 같이 바꿔줘.

화면 가운데에 카드 박스 1개를 두고, 위에서부터:

- 1) 큰 제목: ∞아파트 옵션 신청
- 2) 작은 안내문: "동·호·계약자명·휴대폰 뒤 4자리를 입력해 주세요"
- 3) 입력칸 4개 (각각 라벨 위에)
 - 동 (숫자만, placeholder "예: 101")
 - 호 (숫자만, placeholder "예: 1504")
 - 계약자명 (한글 텍스트)
 - 휴대폰 뒤 4자리 (숫자 4자리만, maxLength 4)
- 4) "다음" 버튼 (가로 100%)
 - 4칸 중 하나라도 비어 있으면 disabled
 - 휴대폰 뒤 4자리는 숫자 4자리가 정확해야 통과

스타일: 모바일에서도 보기 좋게 폭 360px 이내, 둥근 카드, 진한 파란색 버튼.

useState 4개로 입력값 관리. 다음 버튼 클릭은 우선 `console.log({동, 호, 이름, 뒤4})` 만 출력.

변경 후 Apply 해줘.

[스크린샷: Cursor — 변경안 미리보기]

→ **Apply** 클릭.

[object Object]

`http://localhost:5173` 새로고침 (`Cmd + R`)

[스크린샷: 브라우저 — 입력칸 4개 + 회색 "다음" 버튼이 카드 안에 보임]

✅ 입력칸 4개와 회색(=비활성) 버튼이 보임.

[object Object]

- 1.동에 `101`, 호에 `1504`, 이름에 `임동우`, 휴대폰에 `5604` 입력
2. 다음 버튼이 파란색으로 활성화되는지 확인
3. 누르면 브라우저 개발자 도구(F12) **Console** 탭에 `{ dong: "101", ... }` 같은 게 찍히는지

[스크린샷: 4칸 채워진 모습 + Console 출력]

✅ 모두 OK면 **2차 체크포인트** 통과.

[object Object]

폰에서 보면 카드가 너무 작거나 작은 글씨일 수 있어요.

Cursor에 그대로 복사

방금 만든 화면을 모바일(폭 360px 기준)에서도 글씨 16px 이상,
입력칸 높이 44px 이상, 버튼 높이 48px 이상으로 키워줘.
너무 좁은 화면에서도 좌우 16px 여백이 잡히게.

→ **Apply** → 새로그침. 폰 사이즈로 보기:

브라우저 F12 → 위쪽  아이콘 → iPhone 12 같은 모델 선택.

[스크린샷: 개발자도구 — 모바일 시뮬레이터로 본 화면]

[object Object]

지금까지 바뀐 거 인터넷에도 반영:

1. 사이드바 나뉘가지지 아이콘
2. Message: **입주민 입력 화면**
3. ✓ Commit → Sync / Push

1~2분 후 <https://내아이디.github.io/yye-options/> 도 자동 갱신됨.

[스크린샷: 인터넷 주소에서도 입력칸 4개가 보이는 모습]

[object Object]

- ☐ 화면에 입력칸 **4개**가 카드 안에 보인다
 - ☐ 4칸 다 비어 있으면 **다음** 버튼 회색
 - ☐ 4칸 채우면 **다음** 버튼 파란색
 - ☐ 휴대폰 칸은 숫자 4자리만 들어간다
 - ☐ **다음** 누르면 Console 에 입력값 4개가 찍힌다
 - ☐ **GitHub Pages** 인터넷 주소에서도 같은 화면이 보인다
 - ☐ 폰에서도 입력 잘 됨
-

[object Object]

화면	원인	해결
다음 누르면 페이지가 새로고침 됨	<code><form></code> 의 기본 동작	채팅에 "onClick에 e.preventDefault() 넣어줘"
휴대폰에 글자도 들어감	maxLength·숫자 필터 누락	"휴대폰 입력은 숫자만 받게 onChange에서 필터해줘"
모바일에서 키패드 영문 뜸	inputMode 미지정	"휴대폰 입력은 inputMode='numeric' pattern='\d*' 추가해줘"
한글 입력 깨짐	입력 IME	meta charset 정상이면 거의 없음. 새로고침
회색 버튼 안 풀림	trim 안 함	"4칸 trim() 후 비어있는지로 판단" 시키기

 다음 장 미리보기

7장에서 "다음" 버튼이 진짜로 일하게 만듭니다.
누르면 5장에서 만든 검증 함수(`verify_yyc_resident`) 를 호출해서
"통과 / 등록되지 않은 정보" 를 보여 주세요.
이 챕터부터 "인터넷 엑셀이랑 진짜 통신" 시작입니다.

 여기까지 오신 분께

4·5·6장 한 번에 큰 산 3개 넘었어요.
화면(앞)·DB(뒤) 양쪽 토대가 깔린 상태입니다.
한숨 돌리고 7장에서 둘을 연결합니다.

[object Object]

이 장에서 완성하는 것

6장에서 만든 다음 버튼이 일하게 됩니다.

누르면 5장에서 만든 `verify_yyc_resident` 함수가 돌고,

통과면 → 다음 화면(평형 표시),

실패면 → "등록되지 않은 정보입니다" 빨간 메시지.

소요 시간: 약 1.5시간

난이도: ★★★ (Supabase ↔ 화면 첫 연결)

[object Object]

용어	1줄 비유
Supabase Client	"내 사이트가 인터넷 엑셀에 전화 걸 때 쓰는 전화기"
<code>rpc('함수명', { 인자 })</code>	"5장에서 만든 매크로를 이름으로 부르기"
<code>async / await</code>	"전화 걸고 답 올 때까지 기다림"
<code>try / catch</code>	"전화 끊기거나 에러 나면 잡아서 메시지 띄우기"

[object Object]

Cursor 터미널:

```
npm install @supabase/supabase-js
```

✅ 끝나면 빨간 글자 없이 새 줄.

[object Object]

루트(`yyc-options/`)에 `.env.local` 파일을 만들고 4장에서 메모해 둔 값을 넣습니다.

Cursor 왼쪽 파일 트리 빈 곳 우클릭 → New File → `.env.local`

```
VITE_SUPABASE_URL=https://abcd1234.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGc...본인거...
```

[스크린샷: .env.local 파일에 두 줄]

⚠ 이 파일은 Git에 올라가면 안 됨. Cursor가 자동으로 `.gitignore` 에 넣어주지만, 직접 확인:

`.gitignore` 안에 `.env.local` 한 줄이 있는지 보고 없으면 추가.

💡 수정한 .env.local 은 `npm run dev` 다시 켜야 적용됨. Ctrl+C → 다시 `npm run dev`.

[object Object]

🔗 Cursor에 그대로 복사

@App.jsx 에 다음 변경을 적용해줘.

- 1) 파일 맨 위에 Supabase 클라이언트 1개 만들기
 - import { createClient } from '@supabase/supabase-js'
 - const supabase = createClient(import.meta.env.VITE_SUPABASE_URL, import.meta.env.VITE_
- 2) 상태 추가
 - const [loading, setLoading] = useState(false)
 - const [errorMsg, setErrorMsg] = useState('')
 - const [verifiedTypeKey, setVerifiedTypeKey] = useState('')
- 3) 다음 버튼 onClick 동작:
 - setLoading(true), setErrorMsg('')
 - 입력값 정규화: 동·호는 숫자만, 이름은 trim·연속공백 1칸, 휴대폰은 숫자 4자리
 - supabase.rpc('verify_yyc_resident', {
 - p_dong: 동, p_ho: 호, p_contractor: 이름, p_phone_tail: 뒤4
 - })
 - 결과 data가 1줄 이상이면 setVerifiedTypeKey(data[0].type_key)
 - 0줄이면 setErrorMsg('등록되지 않은 정보입니다. 동·호·계약자명·휴대폰 뒤 4자리를 다시 확인해 주세요.')
 - error 가 있으면 setErrorMsg('일시적인 오류가 발생했습니다. 잠시 후 다시 시도해 주세요.')
 - finally setLoading(false)
- 4) 화면 추가:
 - 다음 버튼 위에 errorMsg 가 있으면 빨간 글씨로 보이기
 - loading 중이면 다음 버튼 텍스트가 "확인 중..." 으로
 - verifiedTypeKey 가 채워지면 카드 아래 초록 박스로
"확인 완료 - 평형: {verifiedTypeKey}" 표시 (다음 장에서 진짜 화면으로 교체)

변경 후 Apply.

[object Object]

새로고침 → 4칸 입력 → 다음 클릭.

입력	기대 결과
등록부에 있는 사람 (예: 101 / 1504 / 임동우 / 5604)	초록 박스 "확인 완료 — 평형: 59A"
한 칸이라도 틀림	빨간 글씨 "등록되지 않은 정보입니다"
휴대폰에 3자리만	다음 버튼 회색 (6장에서 막아둠)
인터넷 끄고 시도	빨간 글씨 "일시적인 오류..."

[스크린샷: 통과 — 초록 박스 / 실패 — 빨간 글씨 두 컷]

✅ 두 경우 다 동작하면 7장 통과.

[object Object]

화면	원인	해결
Console에 <code>Invalid API key</code>	<code>.env.local</code> 키 오타	4장에서 다시 복사. dev 서버 재시작
<code>function verify_yyc_resident does not exist</code>	5장 SQL 안 돌렸음	SQL Editor 다시 Run
항상 "등록되지 않은..."	입력값에 공백/한자	정규화 코드 들어갔는지 확인. 등록부 데이터도 확인
CORS 에러	URL 끝에 <code>/</code> 붙임	<code>.env.local</code> 의 URL은 <code>.co</code> 까지만
환경변수 undefined	dev 안 꺾다 꺾음	Ctrl+C → <code>npm run dev</code>

[object Object]

- ☐ `.env.local` 에 URL·anon key 들어 있다
- ☐ `.gitignore` 에 `.env.local` 들어 있다
- ☐ 통과 케이스 → 초록 박스 + 평형 표시
- ☐ 실패 케이스 → 빨간 메시지
- ☐ 통신 중 → 버튼이 "확인 중..."
- ☐ Console에 키 노출되는 에러 없음

📌 다음 장 미리보기

8장에서 그 평형(`type_key`)에 맞는 옵션 화면을 띄웁니다. 옵션 카드 + 가격 + 합계까지.

[object Object]

이 장에서 완성하는 것

7장 통과 후 → 평형(예: 59A)에 맞는 옵션 카드 목록이 나옵니다.

체크박스로 고르면 우측 합계가 실시간 갱신.

소요 시간: 약 2시간

난이도: ★★★

[object Object]

용어	1줄 비유
옵션 카탈로그	"평형별 메뉴판 (이 평형엔 이 옵션·이 가격)"
JSON / JS 객체	"엑셀의 한 행을 그대로 코드로 적은 메모"
map / filter	"엑셀 칼럼 변환·줄 골라내기"
합계 (reduce)	"선택된 가격만 더하기 = SUMIF"

[object Object]

src/optionsCatalog.js 파일을 새로 만들어 그대로 복붙.

```

export const OPTIONS_CATALOG = {
  '59A': [
    { key: 'kitchen-up', label: '주방 상부장 업그레이드', price: 1_200_000 },
    { key: 'floor-grade2', label: '바닥재 2등급', price: 850_000 },
    { key: 'window-blind', label: '전체 창 전동블라인드', price: 2_400_000 },
    { key: 'pantry', label: '팬트리 추가', price: 650_000 },
  ],
  '52A': [
    { key: 'kitchen-up', label: '주방 상부장 업그레이드', price: 1_100_000 },
    { key: 'floor-grade2', label: '바닥재 2등급', price: 780_000 },
    { key: 'pantry', label: '팬트리 추가', price: 620_000 },
  ],
  '48B': [
    { key: 'kitchen-up', label: '주방 상부장 업그레이드', price: 1_000_000 },
    { key: 'floor-grade2', label: '바닥재 2등급', price: 720_000 },
  ],
  '65A': [
    { key: 'kitchen-up', label: '주방 상부장 업그레이드', price: 1_300_000 },
    { key: 'window-blind', label: '전체 창 전동블라인드', price: 2_700_000 },
    { key: 'pantry', label: '팬트리 추가', price: 720_000 },
    { key: 'wine', label: '와인셀러', price: 1_900_000 },
  ],
  '79': [
    { key: 'kitchen-up', label: '주방 상부장 업그레이드', price: 1_500_000 },
    { key: 'window-blind', label: '전체 창 전동블라인드', price: 3_100_000 },
    { key: 'wine', label: '와인셀러', price: 2_100_000 },
    { key: 'aircon', label: '시스템 에어컨 1대 추가', price: 2_800_000 },
  ],
};

```

💡 나중에 옵션 가격이 바뀌면 이 파일만 수정해서 push 하면 끝.

[object Object]

Cursor에 그대로 복사

@App.jsx 에 다음을 추가/변경.

- 1) 위에 `import { OPTIONS_CATALOG } from './optionsCatalog'`
- 2) 상태:
 - `const [step, setStep] = useState('verify')` // 'verify' | 'options' | 'form' | 'done'
 - `const [selected, setSelected] = useState({})` // { key: true } 형태
- 3) 7장의 검증 통과 시 `setStep('options')` 호출. (verifiedTypeKey 도 그대로 유지)
- 4) `step === 'options'` 일 때 화면:
 - 상단: "동 000호 000 님 - 평형 {verifiedTypeKey}" (수정 버튼: `setStep('verify')`)
 - 가운데: `OPTIONS_CATALOG[verifiedTypeKey]` 목록을 카드로 출력
각 카드: 좌측 체크박스 + 옵션명 + 우측 가격(천단위 콤마, 끝에 "원")
체크 시 카드 보더가 파란색·열은 파란 배경
 - 하단(스크롤해도 따라오는 sticky bar):
"선택 {n}개 / 합계 {amount}원" + 큰 "신청서 작성" 버튼
선택 0개면 신청서 작성 버튼 disabled
신청서 작성 클릭 시 `setStep('form')`
- 5) `OPTIONS_CATALOG[verifiedTypeKey]` 가 비어있거나 undefined 면
"이 평형은 옵션 대상이 아닙니다" 안내.

모바일에서도 sticky bar 가 화면 아래 붙도록 `position: sticky bottom: 0.`
변경 후 Apply.

[object Object]

7장 통과 → 옵션 화면이 떠야 함.

1. 카드 체크/해제할 때 합계 즉시 변동
2. 다 해제하면 "신청서 작성" 회색
3. 위쪽 "수정" 누르면 다시 6장 입력 화면
4. 폰 화면(F12 모바일 모드)에서 sticky bar가 아래 붙어 있나

[스크린샷: 옵션 카드 + sticky 합계 바]

✅ 다 OK면 8장 통과.

[object Object]

화면	원인	해결
옵션 화면이 빈 화면	type_key 가 카탈로그에 없음	5장 등록부 type_key 와 카탈로그 키 일치 확인
합계가 NaN	price가 문자열	카탈로그 가격을 숫자(number)로
모바일에서 sticky bar 안 붙음	부모에 overflow: hidden	App 컨테이너 overflow 제거
체크해도 색 변화 없음	className 조건 누락	"체크 시 카드 클래스 'selected' 토글" 추가 지시

[object Object]

- ☐ `optionsCatalog.js` 파일이 만들어졌고 본인 평형도 들어 있다
- ☐ 옵션 카드가 평형별로 다르게 나온다
- ☐ 체크 시 합계가 즉시 갱신
- ☐ 0개 선택 → "신청서 작성" 회색
- ☐ "수정" 누르면 입력 화면으로 복귀
- ☐ 모바일에서 합계 바가 아래 붙어 있다

 다음 장 미리보기

9장에서 "신청서 작성" 클릭 후의 **개인정보 입력 폼**(주민번호 앞6, 주소, 이메일, 비상연락처 등)을 만듭니다.

```
[object Object]
```

이 장에서 완성하는 것

8장에서 "신청서 작성" 누르면 → 개인정보 입력 폼이 뜹니다.

모든 필수 칸을 채워야 "다음(서명)" 버튼이 활성화.

진짜 저장은 11장에서 하고, 이 장에선 화면+검증까지.

소요 시간: 약 2시간

난이도: ★★ ★

[object Object]

용어	1줄 비유
필수 검증	"이 칸 비면 다음 못 감 — 엑셀 데이터 유효성 검사"
정규식 (regex)	"이메일·주민번호 형식 패턴 검사 도구"
PII (개인정보)	"주민번호·전화·주소 같은 민감한 데이터"
마스킹	"주민번호 뒤 표시 안 함"

```
[object Object]
```

항목	형식	필수
계약자명	텍스트	✅ (자동 채움 — 6장 입력값 그대로)
동/호	텍스트	✅ (자동)
평형	텍스트	✅ (자동)
주민등록번호 앞 6자리	숫자 6자리	✅
휴대폰	010-XXXX-XXXX	✅
이메일	이메일 형식	✅
자택주소	텍스트	✅
비상 연락처 (이름 + 전화)	텍스트	❌ 선택
관리자 메모	텍스트	❌ 선택

```
[object Object]
```


Cursor에 그대로 복사

@App.jsx step === 'form' 일 때 화면을 추가해줘.

자동 채움 정보 박스(읽기 전용):

계약자명 / 동·호 / 평형 / 선택 옵션 목록 + 합계

입력 칸:

- 주민등록번호 앞 6자리 (숫자만, maxLength 6)
- 휴대폰 (010-1234-5678 자동 하이픈)
- 이메일 (type=email, 정규식 검증)
- 자택주소 (textarea 1~2줄)
- 비상 연락처 이름 (선택)
- 비상 연락처 전화 (선택, 같은 하이픈 자동)
- 관리자 메모 (선택, textarea)

검증:

- 필수칸 다 채움
- 주민번호 앞 6자리 = 정확히 숫자 6
- 휴대폰 = 010-XXXX-XXXX 패턴
- 이메일 = 일반 정규식

모두 통과해야 "다음 (서명)" 버튼 활성화.

화면 위쪽 작은 안내문:

"입력하신 개인정보는 옵션 신청 처리 목적으로만 사용되며, 동의하신 경우에 한해 저장됩니다."

그 아래 체크박스:

"[필수] 개인정보 수집·이용에 동의합니다."

동의 + 모든 검증 통과 시에만 "다음 (서명)" 활성화.

"다음 (서명)" 클릭 시 setStep('sign') (다음 장에서 만들 단계).

좌측 "이전" 버튼 → setStep('options').

변경 후 Apply.

[object Object]

[스크린샷: 폼 화면 — 자동 채움 박스 + 입력칸 + 동의 체크 + 비활성 다음 버튼]

확인 항목:

- 일부러 이메일 잘못 입력 → 필드 아래 빨간 안내
- 휴대폰에 숫자만 입력해도 자동으로 010- / - 들어가나
- 동의 체크 풀면 다음 버튼 회색
- "이전" 누르면 옵션 화면으로 정확히 돌아옴 (선택 유지)

✅ 다 OK면 9장 통과.

[object Object]

화면	원인	해결
한국 휴대폰 자동 하이픈 안 됨	onChange 없음	"하이픈 자동 삽입 함수 만들어 onChange에서 적용" 추가 지시
이메일 통과되는데 "a@b" 도 ok	정규식 너무 느슨	표준 RFC 정규식으로 교체 요청
다음 버튼 영원히 회색	검증 OR/AND 버그	Cursor에 "현재 활성/비활성 조건 디버그 출력 후 고쳐줘"
뒤로가니 입력값 사라짐	step 바뀌며 state 리셋	폼 state 도 상위에 두고 유지

[object Object]

- ☐ 자동 채움 정보가 정확히 표시됨
- ☐ 주민번호·휴대폰·이메일 검증 동작
- ☐ 동의 체크 풀면 다음 비활성
- ☐ 잘못 입력 시 그 칸 아래 빨간 안내
- ☐ 이전 ↔ 신청서 ↔ 옵션 사이 데이터 유지
- ☐ 모바일 키패드가 숫자칸에선 숫자로 뜸 (`inputMode="numeric"`)

[object Object]

- 주민번호는 앞 6자리만 받기 (뒷자리 받지 마세요).
- 화면 어디서도 "주민번호 전체" 안 보이게.
- 콘솔에 `console.log(form)` 같은 디버그 코드 남기지 말기 → 17장에서 한 번에 정리.

📌 다음 장 미리보기

10장에서 **마우스/손가락 서명 칸**을 만듭니다. 캔버스 1개 + "지우기/다음" 버튼.

서명 이미지가 PNG(base64) 로 만들어져서 11장 저장 단계에서 같이 DB로 갑니다.

```
[object Object]
```

이 장에서 완성하는 것

9장 "다음(서명)" 누르면 → 빈 서명 칸 1개 + 지우기 + 제출 화면.

사용자가 마우스/손가락으로 서명 → PNG 이미지로 변환 → 다음 단계로 넘김.

진짜 저장은 11장에서.

소요 시간: 약 1.5시간

난이도: ★★

```
[object Object]
```

용어	1줄 비유
Canvas (캔버스)	"그림판 — 좌표에 선을 그릴 수 있는 빈 도화지"
pointer 이벤트	"마우스든 손가락이든 똑같이 처리하는 통합 이벤트"
toDataURL('image/png')	"지금 그린 그림을 PNG 문자열로 뽑기 (base64)"
base64	"이미지를 '문자' 로 표현한 긴 글자 = DB에 그대로 저장 가능"

```
[object Object]
```

Cursor에 그대로 복사

src/SignaturePad.jsx 라는 새 파일을 만들어줘.

요구사항:

- props: { value, onChange }
 - value: 현재 서명 dataURL (없으면 빈 문자열)
 - onChange(dataURL): 그릴 때마다 호출
- canvas 폭 100% (최대 480px), 높이 200px, 흰 배경, 1px 회색 보더, 둥근 모서리
- pointerdown / pointermove / pointerup 으로 그리기
 - 손가락(터치) + 마우스 동시 지원 (pointer events)
 - 선 굵기 2.5, 색 검정, lineCap round
 - touch-action: none 으로 스크롤 막기
- 우측 위에 "지우기" 작은 버튼 → 캔버스 clear + onChange('')
- HiDPI 지원: window.devicePixelRatio 적용해서 흐림 방지
- export default SignaturePad

@App.jsx 의 step === 'sign' 화면도 만들어줘.

- 위 안내문 "아래 박스에 서명해 주세요. (마우스 또는 손가락)"
- <SignaturePad value={signature} onChange={setSignature} />
- const [signature, setSignature] = useState('')
- 하단 두 버튼:
 - 좌측 "이전" → setStep('form')
 - 우측 "제출" → signature 가 비어있으면 disabled, 아니면 활성화
(실제 저장은 다음 장에서. 지금은 console.log({ ...form, signature: signature.slice(0,30)}+')

변경 후 Apply.

[object Object]

새로그침 → 6 → 7 → 8 → 9 → 다음(서명) 까지 진행 → 서명 화면.

[스크린샷: 서명 캔버스에 "홍길동" 같은 서명 + 지우기·이전·제출 버튼]

시험	기대
마우스로 그림	검정 선 보임
폰(F12 모바일 모드)에서 손가락 드래그	선 그림, 페이지 스크롤 안 됨
지우기 버튼	캔버스 비워지고 제출 회색으로
제출 버튼	Console에 <code>{ ..., signature: '...' }</code> 찍힘

✅ 다 OK면 10장 통과.

[object Object]

화면	원인	해결
폰에서 그릴 때 화면이 같이 스크롤됨	<code>touch-action</code> 미적용	캔버스에 <code>style={{ touchAction: 'none' }}</code> 추가
선이 흐리고 두 줄로 보임	DPR 미적용	"devicePixelRatio 스케일 적용" 다시 지시
클릭만 하면 점 하나도 안 찍힘	<code>pointerdown</code> 에 한 점 그리기 안 함	첫 점에 작은 dot 찍는 로직 추가
캔버스 좌표가 어긋남	<code>getBoundingClientRect</code> 안 씀	<code>rect.left/top</code> 빼서 좌표 보정
지우기 후에도 제출 활성화	<code>onChange('')</code> 안 보냄	clear 후 <code>onChange('')</code> 호출 확인

[object Object]

- ☐ `src/SignaturePad.jsx` 파일이 생겼다
- ☐ PC·모바일 둘 다 그릴 수 있다
- ☐ 지우기 → 캔버스 비워지고 제출 회색
- ☐ 제출 시 Console 에 PNG dataURL 시작 부분이 찍힌다
- ☐ 이전 → 신청서로 돌아갔다 다시 와도 화면 정상

🔪 다음 장 미리보기

11장에서 "제출"이 진짜 일을 합니다.

신청 데이터 + 서명을 `applications` 테이블에 한 줄로 저장하고,
같은 호수 중복 신청을 막는 `submit_application` RPC도 만듭니다.

```
[object Object]
```

이 장에서 완성하는 것

10장의 "제출" 누르면 → Supabase **applications** 테이블에 한 줄 저장.

같은 동.호의 중복 신청은 거부.

성공 시 **접수번호**가 발급되고 "접수 완료" 화면으로 이동.

소요 시간: 약 2.5시간

난이도: ★★★★★ (지금까지 중 가장 큼 — 그래도 SQL은 복붙)

```
[object Object]
```

용어	1줄 비유
applications 테이블	"신청서 보관함 — 1줄 = 1신청"
JSONB 칼럼	"엑셀 셀 안에 작은 표를 그대로 넣을 수 있는 칸 (옵션 목록)"
접수번호 (receipt_no)	"올해의 N번째 신청이라는 카운터"
submit_application RPC	"1) 중복 검사 2) 접수번호 발급 3) 행 저장 — 한 매크로로"
NOT NULL / CHECK	"이 칸은 비면 안 됨 / 이 패턴이어야 함"

```
[object Object]
```

Supabase **SQL Editor** → **New query** → 아래 전체 복사 → **Run**.

```

-- 1) 신청서 테이블
CREATE TABLE IF NOT EXISTS public.applications (
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    receipt_no text UNIQUE,
    customer_name text NOT NULL,
    dong text NOT NULL,
    ho text NOT NULL,
    unit_type text NOT NULL,
    resident_id_first6 text NOT NULL CHECK (resident_id_first6 ~ '^[0-9]{6}$'),
    phone text NOT NULL,
    email text NOT NULL,
    address text NOT NULL,
    emergency_name text,
    emergency_phone text,
    options jsonb NOT NULL DEFAULT '[]'::jsonb,
    total_amount integer NOT NULL DEFAULT 0,
    signature_data_url text NOT NULL,
    admin_memo text,
    created_at timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT applications_unique_per_unit UNIQUE (dong, ho)
);

-- 2) 접수번호 카운터
CREATE TABLE IF NOT EXISTS public.yyc_receipt_counter (
    id int PRIMARY KEY DEFAULT 1,
    current_no int NOT NULL DEFAULT 0,
    CHECK (id = 1)
);
INSERT INTO public.yyc_receipt_counter (id, current_no)
VALUES (1, 0) ON CONFLICT DO NOTHING;
ALTER TABLE public.yyc_receipt_counter DISABLE ROW LEVEL SECURITY;

-- 3) 다음 접수번호 발급 함수
CREATE OR REPLACE FUNCTION public.next_yyc_receipt_no()
RETURNS text
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
    v_next int;
BEGIN
    UPDATE public.yyc_receipt_counter
        SET current_no = current_no + 1
    WHERE id = 1
    RETURNING current_no INTO v_next;
    RETURN 'YYC-' || to_char(now() AT TIME ZONE 'Asia/Seoul', 'YYYY')
        || '-' || lpad(v_next::text, 4, '0');
END;
$$;
REVOKE ALL ON FUNCTION public.next_yyc_receipt_no() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.next_yyc_receipt_no() TO anon, authenticated;

-- 4) RLS: 외부에서 직접 SELECT/INSERT 금지 (RPC만 허용)
ALTER TABLE public.applications ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON public.applications FROM PUBLIC;

```

```

-- 5) 한 번에 처리하는 submit_application RPC
DROP FUNCTION IF EXISTS public.submit_application(jsonb);

CREATE OR REPLACE FUNCTION public.submit_application(p jsonb)
RETURNS TABLE(receipt_no text)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
    v_dong text := regexp_replace(coalesce(p->>'dong',''), '\D', '', 'g');
    v_ho text := regexp_replace(coalesce(p->>'ho',''), '\D', '', 'g');
    v_receipt text;
BEGIN
    -- 중복 신청 차단
    IF EXISTS (SELECT 1 FROM public.applications WHERE dong = v_dong AND ho = v_ho) THEN
        RAISE EXCEPTION 'duplicate_application' USING ERRCODE = 'P0001';
    END IF;

    -- 접수번호 발급
    v_receipt := public.next_yyc_receipt_no();

    INSERT INTO public.applications(
        receipt_no, customer_name, dong, ho, unit_type,
        resident_id_first6, phone, email, address,
        emergency_name, emergency_phone,
        options, total_amount, signature_data_url, admin_memo
    ) VALUES (
        v_receipt,
        trim(p->>'customer_name'),
        v_dong, v_ho,
        p->>'unit_type',
        p->>'resident_id_first6',
        p->>'phone',
        lower(trim(p->>'email')),
        p->>'address',
        nullif(trim(coalesce(p->>'emergency_name',''), '')), '',
        nullif(trim(coalesce(p->>'emergency_phone',''), '')), '',
        coalesce(p->'options', '[]'::jsonb),
        coalesce((p->>'total_amount')::int, 0),
        p->>'signature_data_url',
        nullif(trim(coalesce(p->>'admin_memo',''), '')), ''
    );

    RETURN QUERY SELECT v_receipt;
END;
$$;
REVOKE ALL ON FUNCTION public.submit_application(jsonb) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.submit_application(jsonb) TO anon, authenticated;

```

✅ **Success. No rows returned** 메시지 확인.

⚠️ 빨간 에러면 메시지를 그대로 Cursor 채팅에 붙이면서:

Cursor에 그대로 복사

Supabase SQL Editor에서 위 SQL 돌렸는데 이 에러 났어. 어디가 문제고 어떻게 고쳐?
[에러 메시지 붙여넣기]

[object Object]

Cursor에 그대로 복사

@App.jsx step === 'sign' 의 "제출" 버튼 onClick 을 다음처럼 바꿔줘.

- 1) 새 상태:
 - const [submitting, setSubmitting] = useState(false)
 - const [submitError, setSubmitError] = useState('')
 - const [receiptNo, setReceiptNo] = useState('')
- 2) 클릭 시:
 - setSubmitting(true), setSubmitError('')
 - selectedOptions = OPTIONS_CATALOG[verifiedTypeKey].filter(o => selected[o.key]).map(o => ({ key: o.key, label: o.label, price: o.price }))
 - totalAmount = selectedOptions.reduce((s,o)=>s+o.price,0)
 - payload = {
 customer_name, dong, ho, unit_type: verifiedTypeKey,
 resident_id_first6, phone, email, address,
 emergency_name, emergency_phone,
 options: selectedOptions, total_amount: totalAmount,
 signature_data_url: signature, admin_memo
 }
 - const { data, error } = await supabase.rpc('submit_application', { p: payload })
 - error 가 있으면:
 - message = error.message || ''
 - if message.includes('duplicate_application')
 - setSubmitError('이미 같은 동·호로 접수된 신청서가 있습니다. 관리자에게 문의해 주세요.')
 - else if message.includes('check') 또는 RLS 관련
 - setSubmitError('입력값에 문제가 있습니다. 다시 확인해 주세요.')
 - else
 - setSubmitError('일시적인 오류가 발생했습니다. 잠시 후 다시 시도해 주세요.')
 - 성공이면 setReceiptNo(data[0].receipt_no), setStep('done')
 - finally setSubmitting(false)
- 3) submitting 중이면 제출 버튼 텍스트 "제출 중..."
submitError 가 있으면 제출 버튼 위에 빨간 글씨
- 4) step === 'done' 화면:
 - 큰 체크 아이콘 + "접수가 완료되었습니다"
 - 접수번호: {receiptNo} (복사 버튼 작게)
 - "처음으로" 버튼 → 모든 state 초기화 후 setStep('verify')

변경 후 Apply.

[object Object]

브라우저 새로고침 → 6 → 7 → 8 → 9 → 10 → 제출.

[스크린샷: "접수가 완료되었습니다 — 접수번호 YYC-2026-0001"]

시험	기대
정상 흐름	접수번호 1개 발급 + done 화면
같은 동·호로 다시 시도	빨간 글씨 "이미 같은 동·호로 접수..."
인터넷 끄고 제출	"일시적인 오류..."
Supabase Table Editor → applications 확인	방금 보낸 1줄 그대로 (서명은 긴 dataURL)

✅ 다 OK면 11장 통과 — MVP 핵심 흐름 완성.

[object Object]

화면	원인	해결
<code>permission denied for function submit_application</code>	GRANT 안 됨	위 SQL 끝의 GRANT 줄 다시 Run
<code>null value in column "..."</code>	폼에서 빈 값 보냄	9장 검증 다시 점검
<code>duplicate key value violates unique constraint</code>	같은 호 + applications_unique_per_unit	정상. 안내문대로
Console에 거대한 base64가 두 번 출력	디버그 로그 남음	console.log 지움
<code>function ... does not exist</code>	RPC 인자 타입 다름	클라에서 <code>{ p: payload }</code> 그대로 보냈는지 확인

[object Object]

- ☐ `applications`, `yyc_receipt_counter` 테이블이 생겼다
- ☐ `submit_application(p_jsonb)` RPC 가 있다
- ☐ 처음 신청 → 접수번호 `YYC-2026-0001`
- ☐ 같은 동·호 재시도 → 빨간 안내
- ☐ Table Editor 에서 신청 1줄이 보인다
- ☐ 처음으로 → 빈 입력 화면으로 정상 복귀

[object Object]

- `applications` 테이블은 **RLS ON + 직접 SELECT 금지**. 외부 사람이 anon 키만 가지고는 1줄도 못 봅니다.

- 화면에 입력한 값은 모두 `submit_application` 매크로 1개로만 들어감.
 - 다음 장(12)에서 자동 누적 엑셀까지 켜고 나면, 17장에서 RLS·관리자 권한을 다시 한 번 단단히 잠급니다.
-

다음 장 미리보기

12장에서 한 줄 신청이 들어올 때마다 **Storage** 의 엑셀 파일에 자동으로 한 줄씩 누적되도록 만듭니다.

Supabase **Edge Function + Database Webhook** 두 개를 처음 만져봅니다 — 그래도 복붙·클릭 위주.

[object Object]

이 장에서 완성하는 것

누가 신청서를 제출하면 → 1초 안에 **Storage** 의 `yyc-contract-live_V1.xlsx` 마지막 줄에 자동 추가.

관리자는 항상 "지금까지 모든 신청" 이 들어찬 엑셀 1개를 받게 됩니다.

소요 시간: 약 2.5시간

난이도: ★★★★★ (Edge Function 첫 등장)

[object Object]

용어	1줄 비유
Storage 버킷	"Supabase 안의 클라우드 폴더"
Edge Function	"DB 옆에 사는 작은 알바 — Supabase가 직접 돌려줌"
Database Webhook	"행이 들어오면 → 알바한테 전화해줘"
<code>xlsx</code> 라이브러리	"엑셀 파일 읽고/쓰는 도구"
WORKBOOK_WEBHOOK_SECRET	"알바실 출입 비밀번호 — 아무나 못 부르게"

전체 흐름:

[사용자 제출] → applications INSERT
→ Database Webhook
→ Edge Function append-workbook-row 호출
→ Storage의 엑셀 다운 → 한 줄 추가 → 다시 업로드

[object Object] [object Object] [object Object]

항목	값
Name	<code>application-workbook</code>
Public bucket	 OFF (비공개)

→ Create bucket.

[스크린샷: New bucket 모달]

[object Object]

엑셀 새 파일 → **A1=순번, B1=접수번호, C1=용도, ... (헤더만)** ← 데이터 줄 없음 으로 저장.
파일명: `yyc-contract-live_V1.xlsx` .

💡 헤더 목록은 코드에서 정확히 일치해야 합니다. 가장 간단한 방법은:
12-3 의 Edge Function 코드를 먼저 본 뒤 그 안의 **HEADERS** 배열을 그대로 엑셀 1행에 가로로 복사.

[스크린샷: 엑셀 파일 — 1행만 헤더, 2행 이후 비어 있음]

[object Object]

Storage → **application-workbook** 클릭 → **Upload file** → 위 엑셀 선택.
업로드 후 파일명이 정확히 **yye-contract-live_V1.xlsx** 인지 확인.

[object Object]

Cursor 왼쪽 트리 → 프로젝트 루트에 폴더 생성:

```
supabase/  
  functions/  
    append-workbook-row/  
      index.ts
```

index.ts 에 그대로 복사:

```

import { serve } from "https://deno.land/std@0.224.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.49.1";
import * as XLSX from "https://esm.sh/xlsx@0.18.5";

const HEADERS = [
  "순번", "접수번호", "용도", "동", "호", "평형",
  "계약자명", "주민등록번호 앞6", "휴대폰", "이메일", "주소",
  "비상연락처(이름)", "비상연락처(전화)",
  "선택옵션", "합계금액", "관리자메모", "접수일시"
];

const SHEET_NAME = "신청서";

const corsHeaders = {
  "access-control-allow-origin": "*",
  "access-control-allow-methods": "POST, OPTIONS",
  "access-control-allow-headers":
    "authorization, x-client-info, apikey, content-type, x-workbook-secret"
};

serve(async (req) => {
  if (req.method === "OPTIONS") return new Response("ok", { headers: corsHeaders });

  try {
    const expected = Deno.env.get("WORKBOOK_WEBHOOK_SECRET") ?? "";
    const got = req.headers.get("x-workbook-secret") ?? "";
    if (!expected || got !== expected) {
      return new Response("unauthorized", { status: 401, headers: corsHeaders });
    }

    const SUPABASE_URL = Deno.env.get("SUPABASE_URL")!;
    const SERVICE_KEY = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!;
    const bucket = Deno.env.get("WORKBOOK_BUCKET") ?? "application-workbook";
    const objectKey = Deno.env.get("WORKBOOK_OBJECT_KEY") ?? "yyc-contract-live_V1.xlsx";
    const templateUrl = Deno.env.get("TEMPLATE_PUBLIC_URL") ?? "";

    const body = await req.json();
    const r = body?.record ?? {};

    const supabase = createClient(SUPABASE_URL, SERVICE_KEY, {
      auth: { persistSession: false }
    });

    let buf: ArrayBuffer | null = null;
    const dl = await supabase.storage.from(bucket).download(objectKey);
    if (dl.data) {
      buf = await dl.data.arrayBuffer();
    } else if (templateUrl) {
      const res = await fetch(templateUrl);
      if (!res.ok) return new Response("422 template fetch failed", { status: 422, headers: corsHeaders });
      buf = await res.arrayBuffer();
    } else {
      return new Response("422 workbook missing and TEMPLATE_PUBLIC_URL unset", { status: 422, headers: corsHeaders });
    }

    const wb = XLSX.read(new Uint8Array(buf), { type: "array" });
    const ws = wb.Sheets[SHEET_NAME] ?? wb.Sheets[wb.SheetNames[0]];
  }
});

```

```

if (!ws) return new Response("422 sheet missing", { status: 422, headers: corsHeaders });

const head = XLSX.utils.sheet_to_json(ws, { header: 1, range: 0 })[0] as string[];
if (!head || HEADERS.some((h, i) => (head[i] ?? "").toString().trim() !== h)) {
  return new Response("422 header mismatch on pivot sheet", { status: 422, headers: corsHeaders });
}

const existing = XLSX.utils.sheet_to_json(ws, { header: 1 }) as any[][];
const nextNo = existing.length;

const optionsLabel = Array.isArray(r.options)
  ? r.options.map((o: any) => `${o.label}(${(o.price ?? 0).toLocaleString()}원)`).join(", ")
  : "";
const createdAt = new Date(r.created_at ?? Date.now())
  .toLocaleString("ko-KR", { timeZone: "Asia/Seoul" });

const newRow = [
  nextNo, r.receipt_no, "음션신청",
  r.dong, r.ho, r.unit_type,
  r.customer_name, r.resident_id_first6,
  r.phone, r.email, r.address,
  r.emergency_name ?? "", r.emergency_phone ?? "",
  optionsLabel, r.total_amount ?? 0,
  r.admin_memo ?? "", createdAt
];

XLSX.utils.sheet_add_aoa(ws, [newRow], { origin: -1 });
const out = XLSX.write(wb, { type: "array", bookType: "xlsx" });

const up = await supabase.storage.from(bucket)
  .upload(objectKey, new Blob([out]), { upsert: true, contentType:
    "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet" });
if (up.error) return new Response("500 " + up.error.message, { status: 500, headers: corsHeaders });

return new Response(JSON.stringify({ ok: true, no: nextNo }), {
  headers: { ...corsHeaders, "content-type": "application/json" }
});
} catch (e) {
  return new Response("500 " + (e?.message ?? String(e)), { status: 500, headers: corsHeaders });
}
});

```

루트에 `supabase/config.toml` 도 만들고:

```

project_id = "yyc-options"

[functions.append-workbook-row]
verify_jwt = false

```

[object Object] [object Object]

Cursor 터미널:

```
npm install -g supabase
```

✅ 끝나면 `supabase --version` 으로 버전 1줄.

[object Object]

```
supabase login
```

→ 브라우저 자동 열림 → Authorize.

[object Object]

4장에서 받은 프로젝트 Ref 가 필요. URL `https://abcd1234.supabase.co` 에서 `abcd1234` 부분.

```
supabase link --project-ref abcd1234
```

[object Object]

이 비밀번호는 12-5의 웹훅에도 똑같이 넣을 거예요. 메모장에 적어두기.

```
supabase secrets set WORKBOOK_WEBHOOK_SECRET="원하는비밀번호16자이상"  
supabase secrets set WORKBOOK_BUCKET="application-workbook"  
supabase secrets set WORKBOOK_OBJECT_KEY="yyc-contract-live_V1.xlsx"
```

[object Object]

```
supabase functions deploy append-workbook-row --no-verify-jwt
```

✅ `Deployed Function` 메시지가 뜨면 OK.

⚠️ `not in a project directory` → `cd /경로/yyc-options` 후 다시.

[object Object] [object Object]

[스크린샷: Database → Webhooks 메뉴]

[object Object]

항목	값
Name	applications-insert-to-workbook
Table	applications
Events	✔ Insert 만
Type	HTTP Request
Method	POST
URL	https://abcd1234.supabase.co/functions/v1/append-workbook-row
HTTP Headers	x-workbook-secret = 12-4(4)에서 정한 비밀번호

→ Confirm

[스크린샷: Webhook 폼 — URL과 헤더 한 줄]

[object Object]

브라우저에서 신청 1건 더 제출.

- 1. Supabase **Storage** → **application-workbook** → **yyc-contract-live_V1.xlsx** "업데이트 시간"이 방금으로 바뀌었나
- 2. 다운로드해서 열기 → 마지막 행에 방금 신청 1줄 추가됐나

[스크린샷: 엑셀 마지막 행에 새 신청 한 줄]

✔ 잘 추가되면 **자동 누적 엑셀** 완성. 14장(관리자 다운로드)에서 이 파일을 안전하게 받게 만듭니다.

[object Object]

화면	원인	해결
Webhook 결과 401	secret 불일치	시크릿/헤더 양쪽 비번 똑같이
422 header mismatch	엑셀 1행 헤더가 코드와 다름	엑셀 1행을 코드의 HEADERS 그대로
422 workbook missing	버킷·파일명 불일치	secrets 의 WORKBOOK_OBJECT_KEY 와 실제 파일명 동일
500 storage upload fail	버킷 권한/이름 오타	application-workbook 정확히
Webhook 도는데 엑셀 안 바뀔	캐시	Storage 새로고침. 한 번 더 신청
not in a project directory	CLI 위치	cd 프로젝트경로
verify_jwt 401	jwt 검사 켜짐	배포 시 --no-verify-jwt 또는 config.toml 확인

[object Object]

- ☐ `application-workbook` 버킷 (비공개) 안에 헤더만 있는 엑셀 1개 업로드
- ☐ `supabase/functions/append-workbook-row/index.ts` 가 있다
- ☐ CLI 로 `secrets set` 3개 + `functions deploy` 완료
- ☐ Webhook 1개 (Insert + URL + 헤더) 등록
- ☐ 새 신청 1건 → 엑셀 마지막 줄 자동 추가
- ☐ **Functions → Logs** 에 200 OK 가 보인다

[object Object]

- 버킷은 **비공개** 상태. 인터넷에서 URL 만으론 못 받음 → 14장에서 **서명 URL** 로 안전하게 받음.
- 웹훅·Edge Function 사이는 `WORKBOOK_WEBHOOK_SECRET` 로 보호.
- Edge Function 안에선 **service_role** 키 를 쓰지만, **외부엔 절대 안 노출** (Supabase 환경변수에만 존재).

 여기까지 오신 분께

10·11·12 묶음으로 신청 한 번 → DB 저장 → 자동 누적 엑셀 까지 흐름이 한 번에 도는 진짜 시스템이 완성됐어요.

[object Object]

이 장에서 완성하는 것

`/?admin=1` 같은 비밀 주소 → 관리자 로그인 화면 → 통과하면
신청자 목록 표 가 보입니다 (접수번호·동·호·이름·평형·금액·일시).
소요 시간: 약 2시간
난이도: ★★★

[object Object]

용어	1줄 비유
Supabase Auth	"Supabase가 만들어주는 로그인 서비스 (비번/이메일·구글 등)"
관리자 계정	"이메일 = <code>admin@admin.com</code> , 비번 한 개 — 한 명만 씀"
세션 (session)	"로그인 후 받은 출입증 — 브라우저에 잠깐 보관"
list_applications RPC	"관리자만 부를 수 있는 매크로 — 신청 전체 목록을 줘"

흐름:

```
[관리자] /?admin=1 접속
  → 이메일·비번 로그인 (Supabase Auth)
  → 세션 토큰 보관
  → list_applications RPC 호출
  → 표 렌더링
```

[object Object] [object Object] [object Object]

항목	값
Email	<code>admin@admin.com</code>
Password	강한 비번 (16자 이상, 메모장에 저장)
Auto Confirm User	☒ 체크

→ Create user.

[스크린샷: Add user 모달]

☒ Users 목록에 `admin@admin.com` 한 줄.

[object Object]

Authentication → Providers → Email

- **Enable Email Signups:**  **OFF**
- Confirm email: 그대로

→ Save.

[스크린샷: Email Provider — Signup OFF]

이렇게 해두면 외부에서 누가 회원가입을 시도해도 못 만듭니다. 관리자 1명만 존재.

[object Object]

SQL Editor → 그대로 복붙 → Run.

```

-- 1) 관리자 화이트리스트 테이블 (이메일 기준)
CREATE TABLE IF NOT EXISTS public.app_admins (
    email text PRIMARY KEY
);
INSERT INTO public.app_admins(email) VALUES ('admin@admin.com')
ON CONFLICT DO NOTHING;
ALTER TABLE public.app_admins ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON public.app_admins FROM PUBLIC;

-- 2) 관리자 여부 검사 함수
CREATE OR REPLACE FUNCTION public.is_admin()
RETURNS boolean
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
    SELECT EXISTS (
        SELECT 1 FROM public.app_admins a
        WHERE a.email = lower(coalesce(auth.jwt()->'email',''))
    );
$$;
REVOKE ALL ON FUNCTION public.is_admin() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.is_admin() TO authenticated;

-- 3) 신청 목록 조회 RPC (관리자만)
DROP FUNCTION IF EXISTS public.list_applications();

CREATE OR REPLACE FUNCTION public.list_applications()
RETURNS TABLE(
    id bigint, receipt_no text, customer_name text,
    dong text, ho text, unit_type text,
    total_amount integer, created_at timestamptz
)
LANGUAGE plpgsql STABLE SECURITY DEFINER SET search_path = public AS $$
BEGIN
    IF NOT public.is_admin() THEN
        RAISE EXCEPTION 'forbidden' USING ERRCODE = '42501';
    END IF;
    RETURN QUERY
        SELECT a.id, a.receipt_no, a.customer_name,
            a.dong, a.ho, a.unit_type,
            a.total_amount, a.created_at
        FROM public.applications a
        ORDER BY a.created_at DESC;
END $$;
REVOKE ALL ON FUNCTION public.list_applications() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.list_applications() TO authenticated;

-- 4) 신청 1건 상세 조회 RPC (14장에서 사용)
DROP FUNCTION IF EXISTS public.get_application(bigint);

CREATE OR REPLACE FUNCTION public.get_application(p_id bigint)
RETURNS SETOF public.applications
LANGUAGE plpgsql STABLE SECURITY DEFINER SET search_path = public AS $$
BEGIN
    IF NOT public.is_admin() THEN
        RAISE EXCEPTION 'forbidden' USING ERRCODE = '42501';
    END IF;
    RETURN QUERY SELECT * FROM public.applications WHERE id = p_id;
END $$;

```

```
REVOKE ALL ON FUNCTION public.get_application(bigint) FROM PUBLIC;  
GRANT EXECUTE ON FUNCTION public.get_application(bigint) TO authenticated;
```

✅ Success 메시지.

[object Object]

Cursor에 그대로 복사

@App.jsx 에 관리자 페이지를 추가해줘.

- 1) URL 검사:
`const isAdminUrl = new URLSearchParams(location.search).get('admin') === '1'`
→ 이게 true면 일반 신청 흐름 대신 관리자 화면을 그린다.
- 2) 관리자 상태:
 - `const [adminSession, setAdminSession] = useState(null)`
 - 시작 시 `supabase.auth.getSession()` 으로 복원
 - `supabase.auth.onAuthStateChange` 로 갱신
- 3) 로그인 화면 (`adminSession` 없을 때):
 - 상단 작은 글씨 "관리자 전용"
 - 이메일·비밀번호 입력 + "로그인" 버튼
 - 이메일은 `lowercase trim` 으로 입력 처리
 - `supabase.auth.signInWithPassword({ email, password })`
 - 실패 시 빨간 안내 (구체 에러 노출 X - "이메일 또는 비밀번호가 올바르지 않습니다.")
- 4) 로그인 성공 후 화면 (`adminSession` 있을 때):
 - 우측 상단 로그인된 이메일 + "로그아웃" 버튼 (`supabase.auth.signOut`)
 - "신청 목록" 제목
 - `supabase.rpc('list_applications')` 호출 → 표 렌더링
칼럼: 접수번호 / 동 / 호 / 평형 / 계약자명 / 합계금액(콤마+원) / 접수일시(YYYY-MM-DD HH:mm)
행 우측에 "상세" 버튼 → 다음 장에서 동작 (지금은 `console.log(id)`)
 - 표 위에 "총 N건" 표시, 새로고침 아이콘 버튼
 - 데이터 없으면 "아직 신청이 없습니다." 안내
- 5) 에러 처리:
 - `rpc` 에러가 `'forbidden'` 이면 "관리자 권한이 없습니다." 안내 + 자동 로그아웃
 - 나머지 에러는 "일시적인 오류" 한 줄

모바일에서도 표 가로 스크롤 가능하게 wrapper에 `overflow-x:auto`.
변경 후 Apply.

[object Object]

<http://localhost:5173/?admin=1> 접속.

시험	기대
관리자 페이지 진입	로그인 폼 표시
잘못된 비번	"이메일 또는 비밀번호가 올바르지 않습니다."
<code>admin@admin.com</code> + 정확한 비번	신청 목록 표
표 위 "총 N건"	11장에서 넣은 신청들이 그대로
로그아웃	다시 로그인 폼
<code>?admin=1</code> 빼고 접속	평소 신청 화면

[스크린샷: 로그인 폼 / 로그인 후 목록 표 두 컷]

[object Object]

평소처럼 commit → push.

`https://내아이디.github.io/yyc-options/?admin=1` 에서도 같은 화면 동작.

✅ 다 OK면 13장 통과.

[object Object]

화면	원인	해결
"Email signups are disabled"	13-2(3)대로 폼 — 정상	가입 X, 로그인만
로그인은 되는데 표가 0건	11장 신청을 안 했음	일반 화면에서 신청 1건
<code>permission denied for function list_applications</code>	GRANT 누락	SQL 끝의 GRANT 줄 다시 Run
'forbidden' 자동 로그아웃	화이트리스트에 이메일 없음	<code>app_admins</code> 에 본인 이메일 INSERT
<code>?admin=1</code> 인데 일반 화면	URL 검사 위치 잘못	useEffect 안에서 step 무시하고 분기
토큰이 새로고침마다 사라짐	<code>persistSession</code> false	기본값 그대로 두기 (true)

[object Object]

- ☐ `admin@admin.com` 계정 1개만 존재, 일반 가입 OFF
- ☐ `app_admins` / `is_admin` / `list_applications` / `get_application` SQL 적용
- ☐ `?admin=1` 진입 시 로그인 폼
- ☐ 로그인 성공 → 목록 표
- ☐ 로그아웃·재로그인 정상

- ☐ 일반 사용자는 `?admin=1` 알아도 비번 없이 못 들어감
-

[object Object]

- 일반인은 anon 키만 알아도 `applications` 테이블에 직접 **SELECT** 불가 (RLS).
 - 목록은 오직 `list_applications` RPC 를 통해서만, 그것도 `is_admin()` 통과해야 결과가 나옴.
 - 17장에서 "관리자 외 INSERT/UPDATE/DELETE 절대 금지" 까지 한 번에 잠급니다.
-

📌 다음 장 미리보기

14장에선 관리자 표의 **상세** 버튼이 일하게 됩니다.

1. 신청서 한 건의 모든 정보(서명 포함) 보기
 2. 누적 엑셀을 안전하게 다운로드 (서명 URL).
-

[object Object]

이 장에서 완성하는 것

표의 "상세" 버튼 → 신청서 한 건의 전체 내용 + 서명 이미지 표시.

그리고 누적 엑셀 을 60초짜리 임시 링크(서명 URL) 로 안전하게 받기.

소요 시간: 약 2시간

난이도: ★★★★★

[object Object]

용어	1줄 비유
상세 (detail) 화면	"신청서 1건을 한 페이지에 다 보여주기"
Signed URL	"60초 동안만 유효한 임시 다운로드 주소"
Edge Function <code>sign-application-workbook</code>	"관리자임을 확인하고 임시 주소를 발급해주는 알바"

[object Object]

`supabase/functions/sign-application-workbook/index.ts` 새로 만들고 복붙.

```

import { serve } from "https://deno.land/std@0.224.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.49.1";

const corsHeaders = {
  "access-control-allow-origin": "*",
  "access-control-allow-methods": "GET, POST, OPTIONS",
  "access-control-allow-headers":
    "authorization, x-client-info, apikey, content-type, x-supabase-api-version, prefer";
};

serve(async (req) => {
  if (req.method === "OPTIONS") return new Response("ok", { headers: corsHeaders });

  try {
    const SUPABASE_URL = Deno.env.get("SUPABASE_URL")!;
    const SERVICE_KEY = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!;
    const bucket = Deno.env.get("WORKBOOK_BUCKET") ?? "application-workbook";
    const objectKey = Deno.env.get("WORKBOOK_OBJECT_KEY") ?? "yyc-contract-live_V1.xlsx";

    const allowedRaw = Deno.env.get("WORKBOOK_RESET_ALLOWED_EMAILS") ?? "";
    const allowed = allowedRaw.split(",").map(s => s.trim().toLowerCase()).filter(Boolean);

    const auth = req.headers.get("authorization") ?? "";
    if (!auth.startsWith("Bearer ")) {
      return new Response("unauthorized", { status: 401, headers: corsHeaders });
    }

    const userClient = createClient(SUPABASE_URL, Deno.env.get("SUPABASE_ANON_KEY")!, {
      global: { headers: { Authorization: auth } },
      auth: { persistSession: false }
    });
    const { data: u, error: uerr } = await userClient.auth.getUser();
    if (uerr || !u?.user) return new Response("unauthorized", { status: 401, headers: corsHeaders });
    const email = (u.user.email ?? "").toLowerCase();
    if (allowed.length && !allowed.includes(email)) {
      return new Response("forbidden", { status: 403, headers: corsHeaders });
    }

    const admin = createClient(SUPABASE_URL, SERVICE_KEY, { auth: { persistSession: false } });
    const expires = 60;
    const { data, error } = await admin.storage.from(bucket).createSignedUrl(objectKey, expires);
    if (error || !data?.signedUrl) {
      return new Response("422 " + (error?.message ?? "no url"), { status: 422, headers: corsHeaders });
    }

    return new Response(JSON.stringify({ url: data.signedUrl, expires_in: expires }), {
      headers: { ...corsHeaders, "content-type": "application/json" }
    });
  } catch (e) {
    return new Response("500 " + (e?.message ?? String(e)), { status: 500, headers: corsHeaders });
  }
});

```

supabase/config.toml 에 추가:

```
[functions.sign-application-workbook]
verify_jwt = false
```

`verify_jwt = false` 로 두지만 함수 안에서 직접 JWT를 확인하므로 안전합니다.

[object Object]

```
supabase secrets set WORKBOOK_RESET_ALLOWED_EMAILS="admin@admin.com"
supabase functions deploy sign-application-workbook --no-verify-jwt
```

✅ 배포 완료 메시지.

[object Object]

Cursor에 그대로 복사

@App.jsx 관리자 화면에 다음을 추가해줘.

- 1) 상태:
 - `const [detail, setDetail] = useState(null)`
 - 표 행의 "상세" 클릭 시 `supabase.rpc('get_application', { p_id: id })`
→ `setDetail(data[0])`
- 2) 상세 패널 (detail 있으면 모달 또는 우측 패널로):
 - 닫기 버튼 → `setDetail(null)`
 - 표시 항목(읽기 전용):
접수번호, 접수일시(KST), 평형, 동/호, 계약자명,
주민번호 앞6 (마스킹: 앞 2자리 + ****),
휴대폰, 이메일, 자택주소,
비상연락처(이름/전화),
선택 옵션 목록 (라벨 + 가격, 합계),
관리자메모,
서명 이미지 (signature_data_url을 `<img>` 로, 최대 폭 320px)
 - 모든 텍스트 출력은 안전하게 처리 (XSS 방지는 17장에서 한 번에 강화)
- 3) 목록 화면 우측 상단에 큰 버튼 "엑셀 다운로드"
클릭 시:

```
const { data, error } = await supabase.functions.invoke('sign-application-workbook')
error 있으면 → alert("다운로드 권한이 없거나 일시 오류입니다.")
성공 시 → window.location.href = data.url
(이 url 은 60초 동안만 유효)
```
- 4) 다운로드 진행 중 버튼 텍스트 "다운로드 준비 중..."

변경 후 Apply.

[object Object]

?admin=1 → 로그인 → 목록 표.

시험	기대
"상세" 버튼 클릭	모달에 한 건 전체 정보 + 서명 그림
주민번호 앞6	12**** 형태로 마스킹
"엑셀 다운로드"	잠시 후 엑셀 자동 저장 — 모든 신청이 들어가 있음
같은 링크 60초 후 다시 열기	만료 (다시 누르면 새 링크)
로그아웃 후 다운로드	401/403 에러 안내

[스크린샷: 상세 모달 + 서명 이미지 / 다운로드된 엑셀 파일 두 컷]

✅ 다 OK면 14장 통과.

[object Object]

화면	원인	해결
Failed to fetch	Edge Function 미배포·CORS	functions deploy 다시. config.toml CORS 헤더 확인
401 unauthorized	로그인 안 된 채 호출	로그인 후 재시도
403 forbidden	WORKBOOK_RESET_ALLOWED_EMAILS 화이트리스트 누락	secrets에 본인 이메일 추가 후 재배포
422 no url	버킷·파일명 불일치	12장에서 정한 이름과 secrets 일치 확인
다운로드 후 1초 만에 만료	시계 차이	expires 60 → 90으로 ↑
서명 이미지 안 보임	잘못된 dataURL	17장에서 safeSignatureSrc 로 검증 강화

[object Object]

- ☐ sign-application-workbook 함수 배포 완료
- ☐ WORKBOOK_RESET_ALLOWED_EMAILS 시크릿에 본인 이메일
- ☐ 표 "상세" → 한 건 전체 표시 + 서명 이미지
- ☐ 주민번호 마스킹 (12****)
- ☐ "엑셀 다운로드" 버튼으로 누적 엑셀 받기 OK
- ☐ 로그아웃 상태에선 다운로드 안 됨

[object Object]

- 버킷이 **비공개** + 60초 서명 URL 이라 외부에 링크가 새도 곧 만료.
 - 함수 안에서 `getUser()` + 화이트리스트 두 번 검사.
 - 화면 표시 시 주민번호는 **앞 6자리 → 앞 2자리 마스킹** 처리.
 - 17장에서 '관리자 외엔 INSERT/UPDATE/DELETE 전부 차단' + 'XSS 방어' 까지 한 번에 잠급니다.
-

다음 장 미리보기

15장에선 시즌이 끝났을 때 **모든 신청을 한 번에 초기화**(DB·접수번호·엑셀)하는 "초기화" 버튼을 안전하게 만듭니다.

[object Object]

이 장에서 완성하는 것

- 1. `applications` 테이블의 모든 줄 삭제
- 2. 접수번호 카운터 0으로
- 3. Storage 의 누적 엑셀을 빈 템플릿으로 리셋
이 셋을 두 번 확인받은 뒤 한 방에.

소요 시간: 약 1.5시간

난이도: ★★★★★

[object Object]

용어	1줄 비유
TRUNCATE	"테이블을 통째로 비우기"
<code>admin_clear_all_applications</code> RPC	"applications 비우기 + 카운터 0 으로"
<code>admin_reset_yyc_receipt_counter</code> RPC	"접수번호 카운터만 0 으로"
<code>reset-application-workbook</code> Edge Function	"Storage 엑셀을 빈 템플릿으로 덮어쓰기"

[object Object]

SQL Editor → 복붙 → Run.

```

-- 1) 신청 전체 삭제 + 접수번호 0
CREATE OR REPLACE FUNCTION public.admin_clear_all_applications()
RETURNS void
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
    IF NOT public.is_admin() THEN
        RAISE EXCEPTION 'forbidden' USING ERRCODE = '42501';
    END IF;
    TRUNCATE TABLE public.applications RESTART IDENTITY;
    TRUNCATE TABLE public.yyc_receipt_counter;
    INSERT INTO public.yyc_receipt_counter(id, current_no) VALUES (1, 0)
    ON CONFLICT DO NOTHING;
END $$;
REVOKE ALL ON FUNCTION public.admin_clear_all_applications() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.admin_clear_all_applications() TO authenticated;

-- 2) 카운터만 0 으로
CREATE OR REPLACE FUNCTION public.admin_reset_yyc_receipt_counter()
RETURNS void
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
    IF NOT public.is_admin() THEN
        RAISE EXCEPTION 'forbidden' USING ERRCODE = '42501';
    END IF;
    TRUNCATE TABLE public.yyc_receipt_counter;
    INSERT INTO public.yyc_receipt_counter(id, current_no) VALUES (1, 0);
END $$;
REVOKE ALL ON FUNCTION public.admin_reset_yyc_receipt_counter() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.admin_reset_yyc_receipt_counter() TO authenticated;

```

✅ Success.

[object Object]

[supabase/functions/reset-application-workbook/index.ts](#) 만들고 복붙.

```

import { serve } from "https://deno.land/std@0.224.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.49.1";

const corsHeaders = {
  "access-control-allow-origin": "*",
  "access-control-allow-methods": "GET, POST, OPTIONS",
  "access-control-allow-headers":
    "authorization, x-client-info, apikey, content-type, x-supabase-api-version, prefer";
};

serve(async (req) => {
  if (req.method === "OPTIONS") return new Response("ok", { headers: corsHeaders });

  try {
    const SUPABASE_URL = Deno.env.get("SUPABASE_URL")!;
    const SERVICE_KEY = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!;
    const bucket = Deno.env.get("WORKBOOK_BUCKET") ?? "application-workbook";
    const objectKey = Deno.env.get("WORKBOOK_OBJECT_KEY") ?? "yyc-contract-live_V1.xlsx";
    const templateUrl = Deno.env.get("TEMPLATE_PUBLIC_URL") ?? "";

    const allowedRaw = Deno.env.get("WORKBOOK_RESET_ALLOWED_EMAILS") ?? "";
    const allowed = allowedRaw.split(",").map(s => s.trim().toLowerCase()).filter(Boolean);

    const auth = req.headers.get("authorization") ?? "";
    if (!auth.startsWith("Bearer ")) {
      return new Response("unauthorized", { status: 401, headers: corsHeaders });
    }

    const userClient = createClient(SUPABASE_URL, Deno.env.get("SUPABASE_ANON_KEY")!, {
      global: { headers: { Authorization: auth } },
      auth: { persistSession: false }
    });
    const { data: u, error: uerr } = await userClient.auth.getUser();
    if (uerr || !u?.user) return new Response("unauthorized", { status: 401, headers: corsHeaders });
    const email = (u.user.email ?? "").toLowerCase();
    if (allowed.length && !allowed.includes(email)) {
      return new Response("forbidden", { status: 403, headers: corsHeaders });
    }

    if (!templateUrl) {
      return new Response("422 TEMPLATE_PUBLIC_URL unset", { status: 422, headers: corsHeaders });
    }
    const tres = await fetch(templateUrl);
    if (!tres.ok) return new Response("422 template fetch failed", { status: 422, headers: corsHeaders });
    const buf = await tres.arrayBuffer();

    const admin = createClient(SUPABASE_URL, SERVICE_KEY, { auth: { persistSession: false } });
    const up = await admin.storage.from(bucket).upload(objectKey, new Blob([buf]), {
      upsert: true,
      contentType: "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
    });
    if (up.error) return new Response("500 " + up.error.message, { status: 500, headers: corsHeaders });

    return new Response(JSON.stringify({ ok: true }), {
      headers: { ...corsHeaders, "content-type": "application/json" }
    });
  } catch (e) {
    return new Response("500 " + (e?.message ?? String(e)), { status: 500, headers: corsHeaders });
  }
});

```



```
}  
});
```

`supabase/config.toml` 에 추가:

```
[functions.reset-application-workbook]  
verify_jwt = false
```

[object Object]

가장 쉬운 방법: 12-2(3) 에서 만든 **헤더만 있는 엑셀** 을 GitHub 리포의 `public/templates/` 폴더에 넣고 commit/push.

GitHub Pages 가 `https://내아이디.github.io/yyc-options/templates/yyc-contract-pivot-template.xlsx` 같은 주소로 자동 서비스합니다.

```
mkdir -p public/templates  
# 위 엑셀을 그 폴더에 yyc-contract-pivot-template.xlsx 이름으로 복사  
git add public/templates  
git commit -m "add reset template"  
git push
```

업로드 완료 후 그 URL 그대로 시크릿:

```
supabase secrets set TEMPLATE_PUBLIC_URL="https://내아이디.github.io/yyc-options/templates/yyc-contract-pivot-template.xlsx"  
supabase functions deploy reset-application-workbook --no-verify-jwt
```

[object Object]

Cursor에 그대로 복사

@App.jsx 관리자 화면 우측 상단(엑셀 다운로드 옆)에 빨간 "초기화" 버튼을 추가해줘.

클릭 시:

1) 1차 confirm:

"이 작업은 다음을 모두 비웁니다:

- 신청 데이터 전체
- 접수번호 카운터 (다시 1번부터)
- 누적 엑셀 (빈 템플릿으로 덮어쓰기)

계속하시겠습니까?"

2) 2차 prompt:

"정말 진행하려면 RESET 을 그대로 입력하세요."

(입력값 trim 후 'RESET' 과 정확히 같아야 진행)

진행 시 순서:

- await supabase.rpc('admin_clear_all_applications')
에러나면 멈추고 안내
- await supabase.rpc('admin_reset_yyc_receipt_counter')
(a가 이미 처리하지만 안전 차원)
- await supabase.functions.invoke('reset-application-workbook')
에러나면 안내 (단, a/b는 이미 진행됨을 명시)

진행 중:

- 버튼 비활성 + "초기화 중..." 텍스트

완료 시:

- 목록 다시 불러오기 (list_applications)
- 알림 "초기화가 완료되었습니다."

변경 후 Apply.

[object Object]

?admin=1 → 로그인 → 신청 1~2건 있는 상태에서 **초기화** 클릭.

단계	기대
1차 confirm	안내문 그대로
2차 입력	RESET 외엔 진행 X
진행 후	표가 "총 0건" / "아직 신청이 없습니다."
신청 1건 더 제출 (일반 화면)	접수번호 다시 YYC-2026-0001 부터
누적 엑셀 다운로드	헤더만 있는 빈 파일 → 새 신청 1줄만 들어 있음

[스크린샷: 초기화 후 빈 목록 + 새 신청 후 0001 접수번호]

✅ 다 OK면 15장 통과 — 시즌 운영 사이클 완성.

[object Object]

화면	원인	해결
<code>forbidden</code>	화이트리스트/관리자 미일치	<code>app_admins</code> · <code>WORKBOOK_RESET_ALLOWED_EMAILS</code> 둘 다 본인 이메일
422 TEMPLATE_PUBLIC_URL unset	secret 누락	위 secrets 명령 다시
422 template fetch failed	템플릿 URL 404	브라우저로 직접 열어보고 다운되는 URL 인지 확인
초기화 후에도 1~155 보임	템플릿에 샘플 데이터 들어 있음	템플릿을 헤더 1줄만 남기고 다시 push
카운터 안 줄어듦	옛날 버전 RPC	위 SQL 다시 Run
초기화 후 신청 시도 → duplicate 에러	같은 동·호 이미 사용 — 다른 동·호로 시도	UNIQUE 제약 때문, 정상

[object Object]

- ☐ `admin_clear_all_applications` , `admin_reset_yyc_receipt_counter` RPC 생성
- ☐ `reset-application-workbook` 함수 배포 + `TEMPLATE_PUBLIC_URL` 시크릿
- ☐ 빈 템플릿 엑셀이 GitHub Pages 로 접근 가능
- ☐ "초기화" 버튼 두 단계 확인 후 동작
- ☐ 초기화 후 새 신청 → 접수번호 0001 부터
- ☐ 초기화 후 다운로드 엑셀 → 헤더만

[object Object]

- 초기화 SQL 은 `is_admin()` 통과해야만 실행 (RLS+SECURITY DEFINER 조합).
- 엑셀 리셋은 Edge Function 안에서 **JWT 확인 + 이메일 화이트리스트** 두 겹.
- 두 단계 confirm/prompt 로 실수 방지.

 **여기까지 오신 분께**
13·14·15 묶음으로 관리자 운영 도구 한 세트가 완성됐어요.

[object Object]

이 장에서 완성하는 것

코드를 `git push` 하면 → 자동으로 빌드 → **GitHub Pages** 갱신.

환경변수(VITE_SUPABASE_URL 등)도 GitHub 비밀 저장소에 안전하게.

소요 시간: 약 1.5시간

난이도: ★★★

[object Object]

용어	1줄 비유
GitHub Actions	"GitHub가 무료로 빌려주는 자동화 알바"
워크플로우 (workflow)	"알바한테 시킬 일 목록 (yaml 한 장)"
Secret / Variable	"공개 안 되는 비밀 메모 / 그냥 메모"
Job / Step	"큰 단위 일 / 그 안의 작은 단계"
gh-pages	"빌드 결과 폴더를 인터넷 주소로 띄워주는 GitHub 기능"

[object Object]

3장에서 "Deploy from a branch" 로 했다면 한 번만 바꿉니다.

[object Object] [object Object]

[스크린샷: Pages 설정 — Source: GitHub Actions]

→ Save (자동 저장될 수도 있음).

[object Object]

`.env.local` 의 두 줄을 GitHub에도 알려줘야 합니다.

[object Object] [object Object]

Name	Secret
<code>VITE_SUPABASE_URL</code>	<code>https://abcd1234.supabase.co</code>
<code>VITE_SUPABASE_ANON_KEY</code>	<code>eyJhbGc...</code> 본인거...

[스크린샷: Actions secrets 화면 — 두 줄 등록됨]

⚠ **service_role 키, Supabase DB 비밀번호, WORKBOOK_WEBHOOK_SECRET 같은 건 여기 절대 넣지 마세요.** 프론트 빌드는 anon 키만 알면 됩니다.

[object Object]

🔗 Cursor에 그대로 복사

프로젝트 루트에 `.github/workflows/pages.yml` 파일을 만들어줘.

요구사항:

- main 브랜치에 push 되면 트리거
- workflow_dispatch (수동 실행)도 가능
- permissions: contents:read, pages:write, id-token:write
- jobs:
 - 1) build:
 - actions/checkout@v4
 - actions/setup-node@v4 (node-version 20, cache: 'npm')
 - npm ci
 - npm run build
 - env:
 - VITE_SUPABASE_URL: `${{ secrets.VITE_SUPABASE_URL }}`
 - VITE_SUPABASE_ANON_KEY: `${{ secrets.VITE_SUPABASE_ANON_KEY }}`
 - VITE_BASE: `"/yyc-options/"`
 - 빌드 후 dist 폴더에 .nojekyll 빈 파일 생성
 - actions/upload-pages-artifact@v3 (path: ./dist)
 - 2) deploy:
 - needs: build
 - environment: name: github-pages, url: `${{ steps.deployment.outputs.page_url }}`
 - actions/deploy-pages@v4 (id: deployment)

그리고 vite.config.js 가 base 옵션을 import.meta.env.BASE_URL 또는 process.env.VITE_BASE 를 쓰게 되어 있는지 확인하고, 안 되어 있으면 그렇게 수정.

변경 후 Apply.

[object Object] [object Object]

- Cursor 사이드바 나뉘가지 → Message: **actions 자동 배포** → ✓ → Sync.

[object Object]

- 방금 push 워크플로우가 노란 점 → 초록 체크로 변하는지 확인.

[스크린샷: Actions 탭 — build / deploy 둘 다 ✅]

[object Object]

- <https://내아이디.github.io/yyc-options/> 에 변경 사항 반영됨.

✅ Push만으로 자동 반영되면 16장 통과.

[object Object]

화면	원인	해결
Actions 빨간 X — <code>npm ci</code> 실패	<code>package-lock.json</code> 누락	로컬에서 <code>npm install</code> 후 lock 파일 commit
사이트가 흰 화면	base 경로 잘못	<code>vite.config.js</code> 에 <code>base: '/yyc-options/'</code>
404 on assets	<code>.nojekyll</code> 누락	yml 의 ".nojekyll 생성" 단계 확인
"Get Pages site failed"	Pages 모드 Branch로 남아있음	16-2 다시
<code>secrets.VITE_SUPABASE_URL</code> 비어 있음	16-3 안 함	secret 등록 후 "Re-run workflow"
빌드는 되는데 anon key 없음	secret 이름 오타	yml의 env 이름과 secret 이름 1:1

[object Object]

- ☐ Pages Source = **GitHub Actions**
- ☐ Repo Secrets 2개 등록 (`VITE_SUPABASE_URL` , `VITE_SUPABASE_ANON_KEY`)
- ☐ `.github/workflows/pages.yml` 생성·push
- ☐ Actions 탭에 ☒ 두 잡 (build, deploy)
- ☐ 코드 수정 → push 만으로 사이트 자동 갱신
- ☐ Actions 로그에 anon/service 키 "값" 이 노출 안 됨

[object Object]

- secret 은 로그에 마스킹되지만, `console.log(env)` 같은 건 절대 코드에 두지 말기.
- `VITE_*` 로 시작하는 건 빌드 후 클라이언트에 그대로 박힘 = 누구나 볼 수 있음. 그래서 `anon` 키만 OK, `service_role` 은 절대 X.
- Webhook secret, DB 비번 등은 GitHub 가 아니라 **Supabase secrets** 에만 둡니다.

📌 다음 장 미리보기

17장에서 **권한(RLS) + XSS** 를 한 번에 단단히 잠급니다. 마지막 보안 검수.

[object Object]

이 장에서 완성하는 것

1. applications 등 모든 표에 관리자 외엔 INSERT/UPDATE/DELETE 차단

2. submit_application / verify_yyc_resident 외 직접 SELECT 전부 차단

3. 관리자 화면의 모든 사용자 입력 출력에 XSS 방어

4. console.log 등 디버그 흔적 제거

소요 시간: 약 2시간

난이도: ★★★★★

[object Object]

용어	1줄 비유
RLS 정책 (POLICY)	"이 줄은 이 조건일 때만 보이게/바꾸게 해"
권한 (GRANT/REVOKE)	"이 사람한테 이 동작 허락/회수"
XSS	"남이 보낸 글 안에 <script> 가 들어 있어 내 화면에서 실행되는 사고"
escape	"<, >, & 같은 글자를 < 식으로 바꿔 그냥 글자처럼 보이게"
Content-Security-Policy	"브라우저에 '외부 스크립트는 다 막아' 명령"

[object Object]

SQL Editor → 그대로 복붙 → Run.

```

-- 0) 외부 직접 권한 회수
REVOKE ALL ON public.applications FROM anon, authenticated;
REVOKE ALL ON public.yyc_resident_registry FROM anon, authenticated;
REVOKE ALL ON public.yyc_receipt_counter FROM anon, authenticated;
REVOKE ALL ON public.app_admins FROM anon, authenticated;

-- 1) RLS 켜기 (이미 켜진 건 무시됨)
ALTER TABLE public.applications ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.yyc_resident_registry ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.app_admins ENABLE ROW LEVEL SECURITY;

-- 2) 기존에 풀려 있던 정책 정리(있다면)
DROP POLICY IF EXISTS applications_select_admin ON public.applications;
DROP POLICY IF EXISTS applications_modify_admin ON public.applications;
DROP POLICY IF EXISTS registry_admin_only ON public.yyc_resident_registry;
DROP POLICY IF EXISTS admins_select_admin ON public.app_admins;

-- 3) applications: 관리자만 SELECT/UPDATE/DELETE (INSERT 는 RPC만 허용 = 정책 없음)
CREATE POLICY applications_select_admin
  ON public.applications FOR SELECT
  TO authenticated
  USING (public.is_admin());

CREATE POLICY applications_modify_admin
  ON public.applications FOR ALL
  TO authenticated
  USING (public.is_admin())
  WITH CHECK (public.is_admin());

-- 4) yyc_resident_registry: 관리자만 (검증은 SECURITY DEFINER RPC 통해서)
CREATE POLICY registry_admin_only
  ON public.yyc_resident_registry FOR ALL
  TO authenticated
  USING (public.is_admin())
  WITH CHECK (public.is_admin());

-- 5) app_admins: 관리자만 SELECT
CREATE POLICY admins_select_admin
  ON public.app_admins FOR SELECT
  TO authenticated
  USING (public.is_admin());

-- 6) 기존 RPC 들의 EXECUTE 권한 재확인
GRANT EXECUTE ON FUNCTION public.verify_yyc_resident(text,text,text,text) TO anon, authenticated;
GRANT EXECUTE ON FUNCTION public.submit_application(jsonb) TO anon, authenticated;
GRANT EXECUTE ON FUNCTION public.next_yyc_receipt_no() TO anon, authenticated;
GRANT EXECUTE ON FUNCTION public.is_admin() TO authenticated;
GRANT EXECUTE ON FUNCTION public.list_applications() TO authenticated;
GRANT EXECUTE ON FUNCTION public.get_application(bigint) TO authenticated;
GRANT EXECUTE ON FUNCTION public.admin_clear_all_applications() TO authenticated;
GRANT EXECUTE ON FUNCTION public.admin_reset_yyc_receipt_counter() TO authenticated;

```

✅ Success.

[object Object] [object Object]

?admin=1 이 아닌 일반 화면 → F12 → Console:

```
const { data, error } = await window.supabase
  ? window.supabase.from('applications').select('*')
  : (await import('@supabase/supabase-js')).createClient(
    import.meta.env.VITE_SUPABASE_URL,
    import.meta.env.VITE_SUPABASE_ANON_KEY
  ).from('applications').select('*');
console.log(data, error);
```

💡 코드 안에 `supabase` 인스턴스를 `window.supabase = supabase` 로 임시 노출시키면 위 명령이 짧아집니다. 시험 후 꼭 제거.

✅ 기대 결과: `data` 가 `[]` 이거나 `error.code === '42501'` (forbidden). 한 줄도 못 봐야 함.

[object Object]

?admin=1 로그인 후 Console:

```
await window.supabase.rpc('list_applications');
```

✅ 신청 목록이 나와야 함.

[object Object]

일반 화면에서 새 신청 1건 → 접수번호 정상 발급, 관리자 표에 한 줄 추가.

✅ 셋 다 OK면 RLS 잠금 통과.

[object Object]

Cursor에 그대로 복사

@App.jsx 에 XSS 방어 유틸을 넣고, 관리자 화면 출력에 적용해줘.

1) 유틸 함수 추가:

```
function escapeHtml(s) {
  return String(s ?? '').replace(/&<>"'`]/g, ch => ({
    '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;', '`': '&#96;',
  }[ch]));
}
const escapeHtmlAttr = escapeHtml;
function safeSignatureSrc(s) {
  const v = String(s ?? '');
  return /^data:image\/(png|jpeg|jpg|gif|webp);base64,[A-Za-z0-9+/=]+$/.test(v) ? v : ''
}
```

2) 관리자 표 / 상세 modal에서 사용자 입력값을 그대로 출력하던 곳을
JSX 의 {value} 출력으로 통일. (React의 JSX 는 기본 escape 됨)
혹시 dangerouslySetInnerHTML 을 쓰는 곳이 있다면 모두 제거하고
꼭 필요한 곳에만 escapeHtml(...) 결과를 사용.

3) 서명 이미지 출력은
`<img src={safeSignatureSrc(detail.signature_data_url)} ... />`
(검증 실패 시 빈 src → 깨진 이미지 placeholder)

4) 로그인 실패 메시지 등 외부에 보여주는 모든 에러 텍스트는
Supabase 가 준 원문(error.message)을 그대로 노출하지 말고
미리 정해둔 한국어 안내 문구로 치환.

5) 콘솔 디버그 정리:
- signature, payload, error 객체를 직접 console.log 하던 줄 모두 제거
- 남기고 싶은 로그는 import.meta.env.DEV 일 때만 출력

변경 후 Apply.

[object Object]

관리자 표/상세에서 진짜 "나쁜 값" 이 들어와도 화면 안 깨지는지.

[object Object] [object Object]

✅ 기대: 알림창 안 뜸. 화면에 그냥 글자 `<img src=x onerror="alert(1)">` 그대로 보여야 함.

[스크린샷: 상세 화면 — 위 문자열이 글자 그대로 표시]

⚠ alert가 뜨면 어딘가 `dangerouslySetInnerHTML` 또는 raw `innerHTML` 가 남은 것. Cursor 채팅에:

Cursor에 그대로 복사

@App.jsx 에서 dangerouslySetInnerHTML 또는 innerHTML 직접 사용 부분 전부 찾아서 JSX 출력 또는 escapeHtml 로 바꿔줘. 변경 후 Apply.

[object Object]

`index.html` `<head>` 안에 한 줄 추가하면 외부 스크립트 자체를 차단.

```
<meta http-equiv="Content-Security-Policy"
  content="default-src 'self';
    img-src 'self' data;;
    connect-src 'self' https://*.supabase.co;
    script-src 'self';
    style-src 'self' 'unsafe-inline';
    base-uri 'self'; frame-ancestors 'none';" />
```

화면이 깨지면 잠시 빼고 17장 다른 항목부터 마치세요. CSP 는 21·22장(부록)에서 다시 다듬어도 됩니다.

[object Object]

- ☐ anon 키로 `from('applications').select('*')` 시도해도 0줄
- ☐ 관리자 로그인 후 `list_applications` 정상
- ☐ 일반 신청 흐름 영향 없음
- ☐ 이름·메모 등에 HTML 넣어 신청해도 화면이 글자로만 보임
- ☐ `console.log(payload/signature/error)` 등 흔적 제거
- ☐ `service_role`, DB 비번, webhook secret 코드/Repo 어디에도 없음

[object Object]

화면	원인	해결
관리자 표 갑자기 0 건	<code>app_admins</code> 에 본인 이메일 없음	INSERT 다시
신청 흐름이 RLS로 막힘	정책 없이 INSERT 시도	<code>submit_application</code> RPC 거치는지 확인
관리자에서 상세 안 열림	<code>get_application</code> 권한 누락	위 GRANT 다시
CSP 적용 후 사이트 깨짐	connect-src 누락	Supabase URL 도메인 추가
Edge Function 호출 막힘	CSP connect-src	<code>https://*.supabase.co</code> 포함 확인
한글 텍스트 깨짐	escape 의 부작용	escape 는 React 출력용 아님. JSX는 자동 처리하므로 유튐은 dangerouslySetInnerHTML 자리에만

[object Object]

항목	상태
일반인 → DB 직접 SELECT	차단 (RLS + REVOKE)
일반인 → DB 직접 INSERT/UPDATE/DELETE	차단 (RPC 만 허용)
일반인 → Storage 엑셀 직접 다운로드	차단 (비공개 버킷)
관리자 → 다운로드	60초 서명 URL
관리자 가입	이메일 가입 OFF + 화이트리스트
입력값 → 화면	escape + safeSignatureSrc
service_role · DB 비번 · webhook secret	Repo·Pages·코드 어디에도 없음
Edge Function 출입	JWT 또는 webhook secret 검사

→ 개인정보 측면에서 **최소한의 외부 노출** 상태에 도달.

 다음 장 미리보기

18장에선 운영자(=본인)가 매주·매월 점검할 **운영 체크리스트와 백업** 을 만듭니다.

[object Object]

이 장에서 완성하는 것

1. 주 1회·월 1회 점검 리스트 1장
2. 신청 데이터 자동 백업 1가지 (Supabase → Storage)
3. 장애 시 첫 30분 행동표 1장

소요 시간: 약 1.5시간

난이도: ★★ (대부분 클릭/복붙)

[object Object]

`docs/OPERATIONS_CHECKLIST.md` 새 파일을 만들고 그대로 복붙.

```
# ㅇㅇ아파트 옵션 신청 - 운영 점검표

## 매주 1회 (월요일 오전)
- [ ] 사이트 접속 확인 (https://내아이다.github.io/yyc-options/)
- [ ] 일반 신청 흐름 1회 테스트(가짜 입주민으로) - done 화면까지
- [ ] `?admin=1` 로그인 → 신청 목록 표시
- [ ] 누적 엑셀 1회 다운로드 → 마지막 행이 방금 신청인지 확인
- [ ] Supabase → Database → Webhooks → 최근 7일 이내 실패 로그 0건
- [ ] Supabase → Edge Functions → Logs 에 5xx 0건

## 매월 1회 (1일)
- [ ] 관리자 비밀번호 외부 메모 잘 있는지 확인 (분실 시 복구 절차)
- [ ] Supabase 사용량 확인 (Database/Storage/Auth 한계 근접 X)
- [ ] GitHub Actions 최근 30일 모두 ✅
- [ ] 백업 파일이 최근 30일치 다 있는지 (18-3 참고)
- [ ] 입주민 등록부 변동 사항 반영 (필요 시 SQL 한 줄)

## 시즌 종료 시
- [ ] 최종 누적 엑셀 안전 위치에 별도 보관 (USB / 사내 드라이브)
- [ ] 관리자 화면 → 초기화 (15장)
- [ ] 새 시즌 옵션 카탈로그(`src/optionsCatalog.js`) 갱신 후 push
```

이 문서는 본인 + 인수인계용. push 해서 GitHub에 같이 두세요.

[object Object]

Cursor에 그대로 복사

docs/INCIDENT_RUNBOOK.md 라는 새 파일을 만들고
다음 7가지 장애 시나리오에 대한 "원인 가설 / 즉시 확인 명령 / 1차 조치 / 사용자 안내 문구" 를
표 형태로 정리해줘. 한국어, 매뉴얼 톤, 비전공자도 따라할 수 있게.

- 1) 사이트가 흰 화면
- 2) 신청 제출 시 "일시적인 오류"
- 3) 관리자 로그인 실패가 모두 "비밀번호 오류"
- 4) 관리자 표 0건 (신청은 들어오는데)
- 5) 누적 엑셀 다운로드 시 401/403
- 6) 누적 엑셀에 새 신청이 안 쌓임 (Webhook 침묵)
- 7) Supabase 가 점검(Outage) 중

각 항목 아래 "체크 후에도 안 풀리면 → 다음 단계" 까지.

→ Apply.

✅ docs/INCIDENT_RUNBOOK.md 가 생기면 한 번 훑어보고, 본인 환경 정보(URL/이메일)를 빈칸에 채워주세요.

[object Object]

Supabase 무료 플랜은 자동 PITR/백업이 제한적입니다.

가장 간단·무료한 방법은 **GitHub Actions** 가 매일 새벽에 누적 엑셀을 받아 다른 폴더에 보관.

[object Object]

Settings → Secrets and variables → Actions → New repository secret

Name	Secret
SUPABASE_SERVICE_ROLE_KEY	Supabase Settings → API 의 service_role 키

⚠ 이건 **비밀 중의 비밀**. 다른 어디에도 적지 말고 GitHub Secret 에만.

[object Object]

Cursor에 그대로 복사

.github/workflows/backup-workbook.yml 을 만들어줘.

트리거:

- schedule: cron '0 18 * * *' # UTC 18:00 = KST 03:00
- workflow_dispatch

한 번에 하나의 Job (backup):

- 1) checkout (default)
- 2) Node 20
- 3) curl 로 Supabase Storage object 다운로드:
URL = \${ secrets.VITE_SUPABASE_URL }/storage/v1/object/application-workbook/yye-con
Header: Authorization: Bearer \${ secrets.SUPABASE_SERVICE_ROLE_KEY }
apikey: \${ secrets.SUPABASE_SERVICE_ROLE_KEY }
저장: backups/yye-contract-live_\$(date -u +%Y%m%d).xlsx
- 4) actions/upload-artifact@v4
name: workbook-backup-\${ github.run_id }
path: backups/
retention-days: 30

코드에는 하드코딩된 키 절대 X. 모두 \${ secrets.* } 만.
변경 후 Apply.

[object Object]

GitHub → Actions → **backup-workbook** → **Run workflow** → 잠시 후 
→ 해당 run 페이지 하단 **Artifacts** 에서 엑셀 다운로드.

[스크린샷: Actions Artifacts — workbook-backup_xxxx]

[object Object]

다음 날 Actions 탭에 자동 실행 1줄 추가됐는지.

 **30일 보관**으로 충분치 않으면, 워크플로우에서 같은 파일을 별도 사설 리포에 push 하는 식으로 늘릴 수 있어요.
(부록 D 에서 다룸)

[object Object]

- ☐ docs/OPERATIONS_CHECKLIST.md 가 리포에 있다
- ☐ docs/INCIDENT_RUNBOOK.md 가 리포에 있다 (빈칸 채움)
- ☐ SUPABASE_SERVICE_ROLE_KEY 가 오직 **GitHub Secret** 에만 존재
- ☐ backup-workbook.yml 수동 실행  , Artifact 다운로드 가능
- ☐ 다음 날 03:00 KST (UTC 18:00) 자동 실행 1건 

[object Object]

화면	원인	해결
401 Unauthorized	service_role 키 오타/누락	Secret 다시 등록
200인데 파일 빈 0바이트	파일명/버킷 오타	URL 의 bucket-objectKey 확인
cron 안 도는데?	계정 무료 한도/지연	UTC 시간 + 5~30분 지연 가능. 다음 날 다시
Artifact 다운 안 됨	보관 기간 만료	retention-days 늘리기
키가 워크플로우 로그에 보일까봐 걱정	secret 은 자동 마스킹	<code>echo \$SECRET</code> 같은 명령만 안 쓰면 됨

[object Object]

- service_role 키는 **Supabase 시크릿 + GitHub Secret + 개인 비밀 보관소** 외 어디에도 두지 않음.
- 백업 파일은 GitHub Actions Artifact (리포 권한자만 다운로드 가능) 에 보관.
- "30일 자동 보관" + "시즌 종료 시 별도 백업"의 2단 구조로 거의 모든 사고 복구 가능.
- 관리자 비밀번호는 분기 1회 변경 권장 (`docs/OPERATIONS_CHECKLIST.md` 매월 항목에 미리 적어 둠).

 **여기까지 오신 분께**
16·17·18 묶음으로 자동 배포 + 보안 최종 잠금 + 운영/백업 까지 완성.

[object Object]

이 장에서 완성하는 것

운영하다 만날 거의 모든 빨간 글씨를 "이 화면이면 → 이거 1줄" 로 빠르게.

새 사고가 나면 이 장 한 번 훑고 → 안 풀리면 18장 INCIDENT_RUNBOOK → 그래도면 Cursor.

사용법: 검색(Cmd+F)에 에러 키워드만 치면 됩니다.

난이도: ★ (지식 사전)

[object Object] [object Object]

- 원인: `vite.config.js` 의 `base` 가 리포 경로와 다름.
- 해결: `base: '/yyc-options/'` . 16-4 기준대로 push.

[object Object]

- 원인: 인터넷 끊김 / Supabase 도메인 차단 / Edge Function 미배포.
- 해결: 인터넷 → Supabase Status → `supabase functions list` 로 함수 존재 확인.

[object Object]

- 원인: `VITE_SUPABASE_ANON_KEY` 오타·미반영.
- 해결: GitHub Secret 다시 + Actions 재실행. 로컬은 dev 재시작.

[object Object]

- 원인: Supabase URL 끝 `/` / Edge Function CORS 헤더 누락.
- 해결: `.env` 의 URL은 `.co` 까지만. 함수 `corsHeaders` 정의 확인.

[object Object]

- 원인: dev 서버 재시작 안 함 / `.env.local` 위치 잘못.
- 해결: 프로젝트 루트에 `.env.local` → Ctrl+C → `npm run dev` .

[object Object]

- 원인: `touch-action` 미설정.
- 해결: 캔버스 `style={{ touchAction: 'none' }}` . 10-4 표 참고.

[object Object]

- 원인: 정규식 단순.
- 해결: 표준 RFC 정규식으로 교체. 9-5 표 참고.

[object Object]

- 원인: trim 미적용 또는 검증 조건 OR/AND 버그.
- 해결: 4칸 모두 trim 후 비어있는지 확인. Cursor에 "현재 활성 조건 디버그 출력 후 고쳐줘".

[object Object] [object Object]

- 원인: `GRANT EXECUTE ... TO authenticated;` 누락.
- 해결: 17-2 SQL 끝부분 GRANT 줄 다시 Run.

[object Object]

- 원인: 인자 타입/개수 다름.
- 해결: 클라가 `{ p: payload }` 로 jsonb 1개를 보내는지 확인. 또는 `DROP FUNCTION` 후 재생성.

[object Object]

- 원인: 같은 동·호 재신청.
- 해결: 정상. 사용자 안내 "이미 같은 동·호로 접수...". 11-3.

[object Object]

- 원인: 폼 검증 빠짐 / 빈 문자열 들어옴.
- 해결: 9장 검증 점검. 빈값은 `nullif(trim(...), '')` 로 NULL 처리.

[object Object]

- 원인: 주민번호 앞6 이 숫자 6 아님.
- 해결: 입력 단계에서 `\d{6}` 만 받기.

[object Object]

- 원인: RLS OFF 또는 정책에 USING (true).
- 해결: 17-2 그대로 다시 Run + 정책에 `is_admin()` 들어갔는지.

[object Object]

- 원인: `app_admins` 에 본인 이메일 없음.
- 해결: `INSERT INTO app_admins(email) VALUES('admin@admin.com');` .

[object Object]

- 원인: 옛 RPC가 카운터 안 올림.
- 해결: 11-2 `next_yyc_receipt_no` 다시 Run.

[object Object]

- 원인: 옛 `admin_clear_all_applications` 사용 중.
- 해결: 15-2 SQL 다시 Run.

[object Object]

- 원인: 카운터 테이블 RLS ON.
- 해결: `ALTER TABLE yyc_receipt_counter DISABLE ROW LEVEL SECURITY;` .

[object Object]

- 원인: 입력값 공백/한자 / 등록부 데이터 다름.
- 해결: 정규화(숫자만/연속공백 1칸) 적용. SQL Editor 에서 직접 호출 시험. 5-5.

[object Object]

- 원인: REVOKE 후 정책 없이 SELECT 시도.
- 해결: 17-2 의 `admins_select_admin` 정책이 있는지.

[object Object] [object Object]

- 원인: `WORKBOOK_WEBHOOK_SECRET` 과 헤더 `x-workbook-secret` 불일치.
- 해결: `supabase secrets set` + Webhook Header 둘 다 같은 값.

[object Object]

- 원인: 엑셀 1행 헤더가 코드의 `HEADERS` 배열과 다름.
- 해결: 엑셀 1행을 12-3 의 `HEADERS` 그대로 다시 입력.

[object Object]

- 원인: 버킷·파일명 오타 / 템플릿 URL 미설정.
- 해결: `WORKBOOK_OBJECT_KEY` = 실제 파일명. `TEMPLATE_PUBLIC_URL` 시크릿 등록.

[object Object]

- 원인: 버킷명 오타 또는 권한.
- 해결: `application-workbook` 정확히. `service_role` 키 정상인지.

[object Object]

- 원인: 캐시 / 화면 새로고침 안 함.
- 해결: Storage 새로고침 → 업데이트 시간 확인.

[object Object]

- 원인: 홈 디렉토리에서 실행.
- 해결: `cd /Users/.../yyc-options` 후 다시.

[object Object]

- 원인: 함수가 JWT 검사 모드인데 webhook 은 토큰 없음.
- 해결: `--no-verify-jwt` 로 배포 또는 `config.toml` 의 `verify_jwt=false` 확인.

[object Object]

- 원인: 미로그인 / 화이트리스트 누락.
- 해결: 관리자 로그인 후 시도. `WORKBOOK_RESET_ALLOWED_EMAILS` 에 본인 이메일.

[object Object]

- 원인: 시계 차이.
- 해결: `expires` 60 → 90~120.

[object Object]

- 원인: 템플릿 자체에 샘플 행.
- 해결: 템플릿 엑셀을 헤더 1줄만 두고 다시 push.

[object Object]

- 원인: Webhook 미연결 / Webhook URL 오타.
- 해결: Webhook URL 끝이 함수명과 정확히 일치하는지.

[object Object] [object Object]

- 원인: 13-2(3)에서 OFF — 정상.

- 해결: 가입 X, 로그인만.

[object Object]

- 해결: Supabase Authentication → Users → 본인 → ... → **Send password recovery**. 받은 메일에서 재설정.

[object Object]

- 해결: 즉시 Supabase **Project Settings → API → Reset** 으로 키 회전. GitHub history 도 `git filter-repo` 등으로 청소(부록 D 참고).

[object Object]

- 원인: `dangerouslySetInnerHTML` 또는 raw `innerHTML` 잔여.
- 해결: 17-4 프롬프트 다시. 모든 출력 JSX 또는 `escapeHtml`.

[object Object]

- 원인: RLS OFF / 정책 USING (true).
- 해결: 17-2 SQL 다시. 17-3 시험.

[object Object] [object Object]

- 원인: `package-lock.json` 누락.
- 해결: `npm install` 후 lock 파일 commit.

[object Object]

- 원인: Pages Source 가 "Branch" 로 남음.
- 해결: Settings → Pages → Source = **GitHub Actions**.

[object Object]

- 원인: `.nojekyll` 누락 / `base` 잘못.
- 해결: 워크플로우의 `.nojekyll` 생성 단계 + `vite.config.js` `base`.

[object Object]

- 원인: GitHub Pages CDN 캐시.
- 해결: Cmd+Shift+R 하드 리프레시. 그래도 안 되면 시크릿 창.

[object Object]

- 원인: `SUPABASE_SERVICE_ROLE_KEY` 오타.
- 해결: Secret 다시 등록 → Re-run.

[object Object]

- 원인: GitHub Free 한도 / 일시 지연.
- 해결: UTC 시간 + 30분 정도 지연 가능. 다음 날 다시. 또는 수동 Run.

[object Object]

- 빨간 메시지 한 줄을 그대로 Cursor 채팅에 붙이고 "이 매뉴얼 19장 표 형식으로 원인.해결 알려줘" 하면, 카탈로그가 자동으로 더 풍부해집니다.

- 새 에러를 만나면 이 파일에 **한 줄 추가**해 두세요. 1년 뒤 본인이 가장 고마워 합니다.
-

다음 장 미리보기

20장은 본 매뉴얼의 **마지막** 본문.

시즌을 시작·끝내고 다른 사람에게 인수인계할 때 그대로 따라할 수 있는 "시즌 운영 가이드".

[object Object]

이 장에서 완성하는 것

- 1. 새 시즌 시작 체크리스트 — 옵션·등록부·테스트 1회씩
- 2. 시즌 종료(클로징) 체크리스트 — 백업 → 초기화 → 카탈로그 갱신
- 3. 인수인계 1장 — 다른 사람이 30분 안에 접수받을 수 있게

소요 시간: 약 1.5시간 (실제 시즌 1번 시뮬레이션 포함)

난이도: ★★ (지금까지 한 일을 묶기만 함)

[object Object]

용어	1줄 비유
시즌	"한 단지 옵션 모집 1번 = 영업 1회차"
카탈로그	"이번 시즌의 메뉴판 (<code>src/optionsCatalog.js</code>)"
등록부	"이번 시즌의 명단 (<code>yyc_resident_registry</code>)"
클로징	"받기 끝 → 결산 → 다음 시즌 준비"

[object Object]

`docs/SEASON_START.md` 새 파일에 그대로 복붙 (체크박스 그대로 사용).

새 시즌 시작 - ∞아파트 옵션 신청

A. 옵션 카탈로그 갱신

- [] `src/optionsCatalog.js` 의 평형·옵션·가격을 이번 시즌 안내문대로 수정
- [] 평형 키(type_key) 가 등록부의 type_key 와 정확히 일치 (예: 59A, 52A, 48B)
- [] commit / push (16장 자동 배포)

B. 입주인 등록부 갱신

- [] 이번 시즌 명단 엑셀 받기
- [] 컬럼 5개로 정리: dong, ho, contractor_name, phone_tail(4자리), type_key
- [] Supabase SQL Editor 에서 INSERT (5-4 형식 그대로)
- [] `SELECT count(\*) FROM yyc\_resident\_registry;` 로 인원 일치 확인

C. 사이트 사전 점검

- [] 일반 신청 1건 테스트 (가짜 입주인) → done 화면
- [] 관리자 로그인 → 그 1건 표·상세·서명 정상
- [] 누적 엑셀 다운로드 → 1줄
- [] 그 후 **초기화 (15장)** 로 깔끔히 비우기 ← ★ 절대 잊지 말기
- [] 다시 누적 엑셀 다운로드 → 0줄(헤더만)

D. 안내문/공지

- [] 입주인에게 보낼 URL: <https://내아이디.github.io/yyc-options/>
- [] 운영 기간 (시작·마감), 문의 연락처를 안내문에 명시
- [] (선택) 단축 URL/QR 만들어 공지문에 첨부

E. 운영 도구 준비

- [] `docs/INCIDENT\_RUNBOOK.md` 빈칸 (URL/이메일/번호) 다시 채움
- [] 관리자 비밀번호 메모 위치 확인
- [] 백업 워크플로우 1회 수동 실행 → Artifact 확인 (18-3)

✅ 위 체크 다 끝나면 "시즌 오픈" 가능 상태.

[object Object]

`docs/SEASON_CLOSE.md` 새 파일에 복붙.

시즌 종료 (클로징) - ∞아파트 옵션 신청

A. 마감 직전 (마감 1시간 전)

- [] 사이트 상단에 마감 안내 (Cursor에 "@App.jsx 상단에 빨간 띠 추가, 'X시 마감 임박' 문구")
- [] 관리자 화면 새로고침 → 신청 건수 메모

B. 마감 후 결산

- [] 관리자 → 누적 엑셀 다운로드 → 파일명 `yyc-시즌YYYY.xlsx` 로 저장
- [] USB / 사내 드라이브 등 안전 위치에 별도 보관 (3중 백업 권장)
- [] 결산표 만들기:
 - 평형별 신청 건수
 - 옵션별 신청 수 / 매출 합계
 - (Cursor 에 엑셀 첨부 후 "이 시트 기반으로 평형·옵션별 합계표 만들어줘")
- [] 회사 양식대로 보고서 작성

C. 다음 시즌 준비

- [] 관리자 화면 → "초기화" (15장) - DB·카운터·누적 엑셀 모두 0
- [] 새 시즌 카탈로그 갱신 (20-2 A)
- [] 새 시즌 등록부 갱신 (20-2 B)
- [] 백업 폴더 `backups/` 의 만료된 파일 정리 (선택)

D. 보안 점검

- [] 관리자 비밀번호 변경 (분기 1회 권장)
- [] GitHub Secrets / Supabase Secrets 변동 사항 없는지
- [] 17장 RLS 정책 그대로 살아있는지 (anon SELECT 시험 1회)

⚠ C의 "초기화" 는 B의 백업이 100% 완료된 뒤에만. 초기화하면 누적 엑셀과 DB가 모두 비워집니다.

[object Object]

`docs/HANDOVER.md` 새 파일에 복붙 → 빈칸을 채워 PDF 한 장으로 출력 가능.

인수인계 - OO아파트 옵션 신청 시스템

1) 한 줄 요약

입주민이 동·호·이름·휴대폰뒤4를 입력하면 등록부와 대조 후
평형별 옵션을 골라 신청하고, 신청은 자동으로 인터넷 엑셀에 누적됩니다.

2) 접속 주소

- 사용자 사이트: https://__GITHUB_ID__.github.io/yyc-options/
- 관리자 페이지: 위 주소 + ?admin=1

3) 관리자 계정

- 이메일: admin@admin.com
- 비밀번호: (별도 보관 - 회사 비밀번호 금고 "옵션신청-관리자")

4) 인프라 계정 (변경/회수 시 절차 ↓)

- GitHub: __GITHUB_ID__
- Supabase 프로젝트: yyc-options (URL: https://__PROJECT_REF__.supabase.co)
- Supabase 결제 카드: (해당 시) 명세
- 도메인: (사용 시) __DOMAIN__

5) 자주 쓰는 작업

- 옵션 가격 바꾸기 → src/optionsCatalog.js 수정 후 push
- 입주민 명단 갱신 → Supabase SQL Editor 에서 INSERT (5-4 형식)
- 신청 결산용 엑셀 받기 → 관리자 페이지 → "엑셀 다운로드"
- 시즌 초기화 → 관리자 페이지 → "초기화" (DB+카운터+엑셀 모두 비움)

6) 사고 시

- 우선 docs/INCIDENT_RUNBOOK.md 의 7가지 시나리오 따라하기
- 그래도 안 풀리면 docs/19장 카탈로그 검색
- 마지막 수단: Cursor 채팅에 에러 메시지 그대로 + "이 매뉴얼 기반으로 원인/해결 알려줘"

7) 정기 작업

- 매주 월요일: docs/OPERATIONS_CHECKLIST.md 의 "매주 1회" 6항목
- 매월 1일: 같은 문서의 "매월 1회" 5항목
- 백업: 매일 03:00 KST 자동 (GitHub Actions → backup-workbook)

8) 절대 하지 말 것

- service_role 키 / DB 비밀번호를 코드·메일·메신저에 적기
- 17장 RLS 정책을 임의로 끄기
- 누적 엑셀(application-workbook 버킷)을 Public 으로 바꾸기
- 백업 없이 "초기화" 누르기

9) 권한 회수 절차 (퇴사·인수인계 시)

1. Supabase Authentication 에서 admin@admin.com 비밀번호 재설정
2. Supabase Project Settings → API → service_role 키 Reset
3. 새 키를 GitHub Secret 에 갱신 (CI 재실행)
4. GitHub 리포 Collaborator 에서 이전 담당자 제거

→ 이 한 장만 있으면 다른 사람이 30분 안에 "접수 받고, 받은 거 보고, 시즌 마감" 까지 할 수 있습니다.

[object Object]

진짜 시즌 전에 혼자서 시즌 1번을 시뮬레이션 하는 걸 강력 추천.

시간	할 일	결과
0:00	20-2 A·B 따라 카탈로그·등록부 가짜 데이터로 갱신	push, INSERT
0:15	일반 화면에서 신청 3건 (가짜 입주민)	접수번호 0001~0003
0:25	관리자 → 표·상세·서명 확인	3건 그대로
0:30	누적 엑셀 다운로드 → 3줄	정상
0:35	백업 워크플로우 수동 실행 → Artifact 다운로드	같은 파일
0:40	20-3 C "초기화" 실행	0건, 헤더만 엑셀
0:45	다시 신청 1건 → 접수번호 0001	통과

✅ 위 6단계가 **에러 없이** 다 통과하면 진짜 시즌 오픈 OK.

[object Object]

- ☐ docs/SEASON_START.md / SEASON_CLOSE.md / HANDOVER.md 가 리포에 있다
- ☐ 인수인계 문서의 빈칸(__GITHUB_ID__ 등)을 채워둠
- ☐ 리허설 시즌 1번 통과
- ☐ 백업 자동 실행이 어제 1건 이상 ✅
- ☐ 본인이 자리를 비울 때 이 3개 문서만 줘도 시즌 운영이 가능

[object Object]

화면	원인	해결
시즌 오픈 직후 "등록되지 않은 정보" 폭주	등록부 INSERT 누락/오타	SQL 카운트와 명단 인원 비교, 빠진 행만 추가
신청은 들어오는데 엑셀에 안 쌓임	Webhook 비활성/오타	Database → Webhooks 활성화·URL 점검 (E-11)
마감 직전 폭증으로 응답 느림	Free 플랜 한계	마감 1시간 전 안내 + 사전 분산 / 필요 시 Pro
마감 후 누적 엑셀 다운 안 됨	service_role 키 회전 직후	Supabase secrets / GitHub secrets 양쪽 갱신
새 시즌 시작 후 옛 신청이 비침	초기화 안 함	20-3 C 실행. 백업이 됐는지 먼저 확인
인수인계 후 첫 운영자가 막힘	비번/접속 정보 부족	docs/HANDOVER.md 빈칸 점검

[object Object]

- 시즌이 끝나면 **백업 → 초기화** 가 보안에도 좋습니다 (오래된 PII를 시스템에 두지 않음).
- 백업본은 **30일 자동 + 시즌 1부 별도 보관**의 2단 구조로 충분.
- 인수인계 시 **service_role** 키와 관리자 비밀번호는 반드시 회전.

 여기까지 오신 분께

0~20장으로 본 매뉴얼의 **본문 전체** 가 끝났습니다.

[object Object]

이 부록의 용도

새 환경에 설치하거나, 의심스러울 때 "지금 SQL 상태가 깨끗한가?" 한 번에 다시 깔 때.

위에서 아래로 순서대로 한 번씩 Run 하면 본 매뉴얼 시점의 DB가 똑같이 만들어집니다.

사용 순서: 4장 프로젝트 생성 → 이 부록 A 한 번 → 14.17장 부분만 다시 확인.

[object Object]

```
-- 깔끔히 다시 시작할 때만. 운영 데이터 모두 사라짐.  
DROP TABLE IF EXISTS public.applications CASCADE;  
DROP TABLE IF EXISTS public.yyc_resident_registry CASCADE;  
DROP TABLE IF EXISTS public.yyc_receipt_counter CASCADE;  
DROP TABLE IF EXISTS public.app_admins CASCADE;  
DROP FUNCTION IF EXISTS public.verify_yyc_resident(text,text,text,text);  
DROP FUNCTION IF EXISTS public.next_yyc_receipt_no();  
DROP FUNCTION IF EXISTS public.submit_application(jsonb);  
DROP FUNCTION IF EXISTS public.is_admin();  
DROP FUNCTION IF EXISTS public.list_applications();  
DROP FUNCTION IF EXISTS public.get_application(bigint);  
DROP FUNCTION IF EXISTS public.admin_clear_all_applications();  
DROP FUNCTION IF EXISTS public.admin_reset_yyc_receipt_counter();
```

[object Object]

```

CREATE TABLE IF NOT EXISTS public.yyc_resident_registry (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  dong text NOT NULL,
  ho text NOT NULL,
  contractor_name text NOT NULL,
  phone_tail text NOT NULL,
  type_key text NOT NULL,
  created_at timestampz NOT NULL DEFAULT now(),
  CONSTRAINT yyc_resident_phone_chk CHECK (phone_tail ~ '^[0-9]{4}$'),
  CONSTRAINT yyc_resident_unique UNIQUE (dong, ho, contractor_name, phone_tail)
);
ALTER TABLE public.yyc_resident_registry ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON public.yyc_resident_registry FROM PUBLIC;

CREATE OR REPLACE FUNCTION public.verify_yyc_resident(
  p_dong text, p_ho text, p_contractor text, p_phone_tail text
)
RETURNS TABLE(type_key text)
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
  SELECT r.type_key FROM public.yyc_resident_registry r
  WHERE r.dong = regexp_replace(coalesce(p_dong,''), '\D','','g')
    AND r.ho = regexp_replace(coalesce(p_ho,''), '\D','','g')
    AND r.contractor_name = trim(regexp_replace(coalesce(p_contractor,''), '\s+', ' ','g'))
    AND r.phone_tail = regexp_replace(coalesce(p_phone_tail,''), '\D','','g')
  LIMIT 1;
$$;
REVOKE ALL ON FUNCTION public.verify_yyc_resident(text,text,text,text) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.verify_yyc_resident(text,text,text,text) TO anon, authenticat

```

[object Object]

```

CREATE TABLE IF NOT EXISTS public.applications (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  receipt_no text UNIQUE,
  customer_name text NOT NULL,
  dong text NOT NULL,
  ho text NOT NULL,
  unit_type text NOT NULL,
  resident_id_first6 text NOT NULL CHECK (resident_id_first6 ~ '^[0-9]{6}$'),
  phone text NOT NULL,
  email text NOT NULL,
  address text NOT NULL,
  emergency_name text,
  emergency_phone text,
  options jsonb NOT NULL DEFAULT '[]'::jsonb,
  total_amount integer NOT NULL DEFAULT 0,
  signature_data_url text NOT NULL,
  admin_memo text,
  created_at timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT applications_unique_per_unit UNIQUE (dong, ho)
);

CREATE TABLE IF NOT EXISTS public.yyc_receipt_counter (
  id int PRIMARY KEY DEFAULT 1,
  current_no int NOT NULL DEFAULT 0,
  CHECK (id = 1)
);
INSERT INTO public.yyc_receipt_counter(id, current_no) VALUES (1, 0)
ON CONFLICT DO NOTHING;
ALTER TABLE public.yyc_receipt_counter DISABLE ROW LEVEL SECURITY;

CREATE OR REPLACE FUNCTION public.next_yyc_receipt_no()
RETURNS text LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE v_next int;
BEGIN
  UPDATE public.yyc_receipt_counter
    SET current_no = current_no + 1
  WHERE id = 1
  RETURNING current_no INTO v_next;
  RETURN 'YYC-' || to_char(now() AT TIME ZONE 'Asia/Seoul', 'YYYY')
    || '-' || lpad(v_next::text, 4, '0');
END $$;
REVOKE ALL ON FUNCTION public.next_yyc_receipt_no() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.next_yyc_receipt_no() TO anon, authenticated;

ALTER TABLE public.applications ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON public.applications FROM PUBLIC;

CREATE OR REPLACE FUNCTION public.submit_application(p jsonb)
RETURNS TABLE(receipt_no text)
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
  v_dong text := regexp_replace(coalesce(p->>'dong',''), '\D','','g');
  v_ho text := regexp_replace(coalesce(p->>'ho',''), '\D','','g');
  v_receipt text;
BEGIN
  IF EXISTS (SELECT 1 FROM public.applications WHERE dong = v_dong AND ho = v_ho) THEN
    RAISE EXCEPTION 'duplicate_application' USING ERRCODE = 'P0001';

```

```

END IF;
v_receipt := public.next_yyc_receipt_no();
INSERT INTO public.applications(
    receipt_no, customer_name, dong, ho, unit_type,
    resident_id_first6, phone, email, address,
    emergency_name, emergency_phone,
    options, total_amount, signature_data_url, admin_memo
) VALUES (
    v_receipt,
    trim(p->>'customer_name'),
    v_dong, v_ho,
    p->>'unit_type',
    p->>'resident_id_first6',
    p->>'phone',
    lower(trim(p->>'email')),
    p->>'address',
    nullif(trim(coalesce(p->>'emergency_name','')), ''),
    nullif(trim(coalesce(p->>'emergency_phone','')), ''),
    coalesce(p->'options', '[]'::jsonb),
    coalesce((p->>'total_amount')::int, 0),
    p->>'signature_data_url',
    nullif(trim(coalesce(p->>'admin_memo','')), '')
);
RETURN QUERY SELECT v_receipt;
END $$;
REVOKE ALL ON FUNCTION public.submit_application(jsonb) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.submit_application(jsonb) TO anon, authenticated;

```

[object Object]

```

CREATE TABLE IF NOT EXISTS public.app_admins ( email text PRIMARY KEY );
INSERT INTO public.app_admins(email) VALUES ('admin@admin.com')
ON CONFLICT DO NOTHING;
ALTER TABLE public.app_admins ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON public.app_admins FROM PUBLIC;

CREATE OR REPLACE FUNCTION public.is_admin()
RETURNS boolean LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
    SELECT EXISTS(
        SELECT 1 FROM public.app_admins a
        WHERE a.email = lower(coalesce(auth.jwt()->>'email',''))
    );
$$;
REVOKE ALL ON FUNCTION public.is_admin() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.is_admin() TO authenticated;

CREATE OR REPLACE FUNCTION public.list_applications()
RETURNS TABLE(
    id bigint, receipt_no text, customer_name text,
    dong text, ho text, unit_type text,
    total_amount integer, created_at timestamptz
)
LANGUAGE plpgsql STABLE SECURITY DEFINER SET search_path = public AS $$
BEGIN
    IF NOT public.is_admin() THEN RAISE EXCEPTION 'forbidden' USING ERRCODE='42501'; END IF;
    RETURN QUERY
        SELECT a.id, a.receipt_no, a.customer_name,
            a.dong, a.ho, a.unit_type,
            a.total_amount, a.created_at
        FROM public.applications a
        ORDER BY a.created_at DESC;
END $$;
REVOKE ALL ON FUNCTION public.list_applications() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.list_applications() TO authenticated;

CREATE OR REPLACE FUNCTION public.get_application(p_id bigint)
RETURNS SETOF public.applications
LANGUAGE plpgsql STABLE SECURITY DEFINER SET search_path = public AS $$
BEGIN
    IF NOT public.is_admin() THEN RAISE EXCEPTION 'forbidden' USING ERRCODE='42501'; END IF;
    RETURN QUERY SELECT * FROM public.applications WHERE id = p_id;
END $$;
REVOKE ALL ON FUNCTION public.get_application(bigint) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.get_application(bigint) TO authenticated;

```

[object Object]


```
CREATE OR REPLACE FUNCTION public.admin_clear_all_applications()
RETURNS void LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
    IF NOT public.is_admin() THEN RAISE EXCEPTION 'forbidden' USING ERRCODE='42501'; END IF;
    TRUNCATE TABLE public.applications RESTART IDENTITY;
    TRUNCATE TABLE public.yyc_receipt_counter;
    INSERT INTO public.yyc_receipt_counter(id, current_no) VALUES (1, 0)
    ON CONFLICT DO NOTHING;
END $$;
REVOKE ALL ON FUNCTION public.admin_clear_all_applications() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.admin_clear_all_applications() TO authenticated;

CREATE OR REPLACE FUNCTION public.admin_reset_yyc_receipt_counter()
RETURNS void LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
    IF NOT public.is_admin() THEN RAISE EXCEPTION 'forbidden' USING ERRCODE='42501'; END IF;
    TRUNCATE TABLE public.yyc_receipt_counter;
    INSERT INTO public.yyc_receipt_counter(id, current_no) VALUES (1, 0);
END $$;
REVOKE ALL ON FUNCTION public.admin_reset_yyc_receipt_counter() FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.admin_reset_yyc_receipt_counter() TO authenticated;
```

[object Object]

```

REVOKE ALL ON public.applications          FROM anon, authenticated;
REVOKE ALL ON public.yyc_resident_registry FROM anon, authenticated;
REVOKE ALL ON public.yyc_receipt_counter   FROM anon, authenticated;
REVOKE ALL ON public.app_admins            FROM anon, authenticated;

ALTER TABLE public.applications          ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.yyc_resident_registry ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.app_admins            ENABLE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS applications_select_admin ON public.applications;
DROP POLICY IF EXISTS applications_modify_admin ON public.applications;
DROP POLICY IF EXISTS registry_admin_only      ON public.yyc_resident_registry;
DROP POLICY IF EXISTS admins_select_admin      ON public.app_admins;

CREATE POLICY applications_select_admin
  ON public.applications FOR SELECT TO authenticated
  USING (public.is_admin());

CREATE POLICY applications_modify_admin
  ON public.applications FOR ALL TO authenticated
  USING (public.is_admin()) WITH CHECK (public.is_admin());

CREATE POLICY registry_admin_only
  ON public.yyc_resident_registry FOR ALL TO authenticated
  USING (public.is_admin()) WITH CHECK (public.is_admin());

CREATE POLICY admins_select_admin
  ON public.app_admins FOR SELECT TO authenticated
  USING (public.is_admin());

GRANT EXECUTE ON FUNCTION public.verify_yyc_resident(text,text,text,text) TO anon, authenticated;
GRANT EXECUTE ON FUNCTION public.submit_application(jsonb)                TO anon, authenticated;
GRANT EXECUTE ON FUNCTION public.next_yyc_receipt_no()                    TO anon, authenticated;
GRANT EXECUTE ON FUNCTION public.is_admin()                              TO authenticated;
GRANT EXECUTE ON FUNCTION public.list_applications()                     TO authenticated;
GRANT EXECUTE ON FUNCTION public.get_application(bigint)                 TO authenticated;
GRANT EXECUTE ON FUNCTION public.admin_clear_all_applications()           TO authenticated;
GRANT EXECUTE ON FUNCTION public.admin_reset_yyc_receipt_counter()        TO authenticated;

```

[object Object]

```
-- 신청 건수 / 평형별
SELECT count(*) FROM public.applications;
SELECT unit_type, count(*) FROM public.applications GROUP BY 1 ORDER BY 1;

-- 옵션별 매출 합계 (jsonb 풀어 보기)
SELECT o->>'label' AS option_label,
       sum((o->>'price')::int) AS total
FROM public.applications, jsonb_array_elements(options) o
GROUP BY 1 ORDER BY 2 DESC;

-- 등록부 인원
SELECT count(*) FROM public.yyc_resident_registry;

-- 현재 접수번호
SELECT current_no FROM public.yyc_receipt_counter WHERE id = 1;

-- 관리자 명단
SELECT * FROM public.app_admins;

-- (관리자로 로그인 후) 본인 인식되는지
SELECT public.is_admin();
```

[object Object]

이 부록의 용도

함수 3개를 한 번에 다시 깔거나, 다른 Supabase 프로젝트로 이식할 때.

모두 `supabase/functions/<이름>/index.ts` 위치.

[object Object]

```
project_id = "yyc-options"

[functions.append-workbook-row]
verify_jwt = false

[functions.sign-application-workbook]
verify_jwt = false

[functions.reset-application-workbook]
verify_jwt = false
```

[object Object]

```

import { serve } from "https://deno.land/std@0.224.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.49.1";
import * as XLSX from "https://esm.sh/xlsx@0.18.5";

const HEADERS = [
  "순번", "접수번호", "용도", "동", "호", "평형",
  "계약자명", "주민등록번호 앞6", "휴대폰", "이메일", "주소",
  "비상연락처(이름)", "비상연락처(전화)",
  "선택옵션", "합계금액", "관리자메모", "접수일시"
];

const SHEET_NAME = "신청서";

const corsHeaders = {
  "access-control-allow-origin": "*",
  "access-control-allow-methods": "POST, OPTIONS",
  "access-control-allow-headers":
    "authorization, x-client-info, apikey, content-type, x-workbook-secret"
};

serve(async (req) => {
  if (req.method === "OPTIONS") return new Response("ok", { headers: corsHeaders });
  try {
    const expected = Deno.env.get("WORKBOOK_WEBHOOK_SECRET") ?? "";
    const got = req.headers.get("x-workbook-secret") ?? "";
    if (!expected || got !== expected)
      return new Response("unauthorized", { status: 401, headers: corsHeaders });

    const SUPABASE_URL = Deno.env.get("SUPABASE_URL")!;
    const SERVICE_KEY = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!;
    const bucket = Deno.env.get("WORKBOOK_BUCKET") ?? "application-workbook";
    const objectKey = Deno.env.get("WORKBOOK_OBJECT_KEY") ?? "yyc-contract-live_V1.xlsx";
    const templateUrl = Deno.env.get("TEMPLATE_PUBLIC_URL") ?? "";

    const body = await req.json();
    const r = body?.record ?? {};
    const supabase = createClient(SUPABASE_URL, SERVICE_KEY, { auth: { persistSession: false } });

    let buf: ArrayBuffer | null = null;
    const dl = await supabase.storage.from(bucket).download(objectKey);
    if (dl.data) buf = await dl.data.arrayBuffer();
    else if (templateUrl) {
      const res = await fetch(templateUrl);
      if (!res.ok) return new Response("422 template fetch failed", { status: 422, headers: corsHeaders });
      buf = await res.arrayBuffer();
    } else {
      return new Response("422 workbook missing and TEMPLATE_PUBLIC_URL unset", { status: 422, headers: corsHeaders });
    }

    const wb = XLSX.read(new Uint8Array(buf), { type: "array" });
    const ws = wb.Sheets[SHEET_NAME] ?? wb.Sheets[wb.SheetNames[0]];
    if (!ws) return new Response("422 sheet missing", { status: 422, headers: corsHeaders });

    const head = (XLSX.utils.sheet_to_json(ws, { header: 1, range: 0 }))[0] ?? [];
    if (HEADERS.some((h, i) => (head[i] ?? "").toString().trim() !== h))
      return new Response("422 header mismatch on pivot sheet", { status: 422, headers: corsHeaders });

    const existing = XLSX.utils.sheet_to_json(ws, { header: 1 }) as any[][];
  }
});

```

```

const nextNo = existing.length;

const optionsLabel = Array.isArray(r.options)
  ? r.options.map((o: any) => `${o.label}(${o.price ?? 0}.toLocaleString())원`).join(", ")
  : "";
const createdAt = new Date(r.created_at ?? Date.now())
  .toLocaleString("ko-KR", { timeZone: "Asia/Seoul" });

XLSX.utils.sheet_add_aoa(ws, [[
  nextNo, r.receipt_no, "옵션신청",
  r.dong, r.ho, r.unit_type,
  r.customer_name, r.resident_id_first6,
  r.phone, r.email, r.address,
  r.emergency_name ?? "", r.emergency_phone ?? "",
  optionsLabel, r.total_amount ?? 0,
  r.admin_memo ?? "", createdAt
]], { origin: -1 });

const out = XLSX.write(wb, { type: "array", bookType: "xlsx" });
const up = await supabase.storage.from(bucket).upload(objectKey, new Blob([out]), {
  upsert: true,
  contentType: "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
});
if (up.error) return new Response("500 " + up.error.message, { status: 500, headers: corsHe

return new Response(JSON.stringify({ ok: true, no: nextNo }), {
  headers: { ...corsHeaders, "content-type": "application/json" }
});
} catch (e) {
  return new Response("500 " + (e?.message ?? String(e)), { status: 500, headers: corsHeaders
}
});

```

[object Object]

```

import { serve } from "https://deno.land/std@0.224.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.49.1";

const corsHeaders = {
  "access-control-allow-origin": "*",
  "access-control-allow-methods": "GET, POST, OPTIONS",
  "access-control-allow-headers":
    "authorization, x-client-info, apikey, content-type, x-supabase-api-version, prefer";
};

serve(async (req) => {
  if (req.method === "OPTIONS") return new Response("ok", { headers: corsHeaders });
  try {
    const SUPABASE_URL = Deno.env.get("SUPABASE_URL")!;
    const SERVICE_KEY = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!;
    const bucket = Deno.env.get("WORKBOOK_BUCKET") ?? "application-workbook";
    const objectKey = Deno.env.get("WORKBOOK_OBJECT_KEY") ?? "yyc-contract-live_V1.xlsx";
    const allowed = (Deno.env.get("WORKBOOK_RESET_ALLOWED_EMAILS") ?? "")
      .split(",")
      .map(s => s.trim().toLowerCase())
      .filter(Boolean);

    const auth = req.headers.get("authorization") ?? "";
    if (!auth.startsWith("Bearer "))
      return new Response("unauthorized", { status: 401, headers: corsHeaders });

    const userClient = createClient(SUPABASE_URL, Deno.env.get("SUPABASE_ANON_KEY")!, {
      global: { headers: { Authorization: auth } },
      auth: { persistSession: false }
    });
    const { data: u, error: uerr } = await userClient.auth.getUser();
    if (uerr || !u?.user) return new Response("unauthorized", { status: 401, headers: corsHeaders });
    const email = (u.user.email ?? "").toLowerCase();
    if (allowed.length && !allowed.includes(email))
      return new Response("forbidden", { status: 403, headers: corsHeaders });

    const admin = createClient(SUPABASE_URL, SERVICE_KEY, { auth: { persistSession: false } });
    const expires = 60;
    const { data, error } = await admin.storage.from(bucket).createSignedUrl(objectKey, expires);
    if (error || !data?.signedUrl)
      return new Response("422 " + (error?.message ?? "no url"), { status: 422, headers: corsHeaders });

    return new Response(JSON.stringify({ url: data.signedUrl, expires_in: expires }), {
      headers: { ...corsHeaders, "content-type": "application/json" }
    });
  } catch (e) {
    return new Response("500 " + (e?.message ?? String(e)), { status: 500, headers: corsHeaders });
  }
});

```

[object Object]

```

import { serve } from "https://deno.land/std@0.224.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.49.1";

const corsHeaders = {
  "access-control-allow-origin": "*",
  "access-control-allow-methods": "GET, POST, OPTIONS",
  "access-control-allow-headers":
    "authorization, x-client-info, apikey, content-type, x-supabase-api-version, prefer"
};

serve(async (req) => {
  if (req.method === "OPTIONS") return new Response("ok", { headers: corsHeaders });
  try {
    const SUPABASE_URL = Deno.env.get("SUPABASE_URL")!;
    const SERVICE_KEY = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!;
    const bucket = Deno.env.get("WORKBOOK_BUCKET") ?? "application-workbook";
    const objectKey = Deno.env.get("WORKBOOK_OBJECT_KEY") ?? "yyc-contract-live_V1.xlsx";
    const templateUrl = Deno.env.get("TEMPLATE_PUBLIC_URL") ?? "";
    const allowed = (Deno.env.get("WORKBOOK_RESET_ALLOWED_EMAILS") ?? "")
      .split(",").map(s => s.trim().toLowerCase()).filter(Boolean);

    const auth = req.headers.get("authorization") ?? "";
    if (!auth.startsWith("Bearer "))
      return new Response("unauthorized", { status: 401, headers: corsHeaders });

    const userClient = createClient(SUPABASE_URL, Deno.env.get("SUPABASE_ANON_KEY")!, {
      global: { headers: { Authorization: auth } },
      auth: { persistSession: false }
    });
    const { data: u, error: uerr } = await userClient.auth.getUser();
    if (uerr || !u?.user) return new Response("unauthorized", { status: 401, headers: corsHeaders });
    const email = (u.user.email ?? "").toLowerCase();
    if (allowed.length && !allowed.includes(email))
      return new Response("forbidden", { status: 403, headers: corsHeaders });

    if (!templateUrl) return new Response("422 TEMPLATE_PUBLIC_URL unset", { status: 422, headers: corsHeaders });
    const tres = await fetch(templateUrl);
    if (!tres.ok) return new Response("422 template fetch failed", { status: 422, headers: corsHeaders });
    const buf = await tres.arrayBuffer();

    const admin = createClient(SUPABASE_URL, SERVICE_KEY, { auth: { persistSession: false } });
    const up = await admin.storage.from(bucket).upload(objectKey, new Blob([buf]), {
      upsert: true,
      contentType: "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
    });
    if (up.error) return new Response("500 " + up.error.message, { status: 500, headers: corsHeaders });

    return new Response(JSON.stringify({ ok: true }), {
      headers: { ...corsHeaders, "content-type": "application/json" }
    });
  } catch (e) {
    return new Response("500 " + (e?.message ?? String(e)), { status: 500, headers: corsHeaders });
  }
});

```


[object Object]

```
cd /경로/yyc-options
supabase link --project-ref __PROJECT_REF__

# 시크릿 (한 번에)
supabase secrets set \
  WORKBOOK_WEBHOOK_SECRET="원하는비밀번호16자이상" \
  WORKBOOK_BUCKET="application-workbook" \
  WORKBOOK_OBJECT_KEY="yyc-contract-live_V1.xlsx" \
  TEMPLATE_PUBLIC_URL="https://__GITHUB_ID__.github.io/yyc-options/templates/yyc-contract-pivot" \
  WORKBOOK_RESET_ALLOWED_EMAILS="admin@admin.com"

# 함수 3개 배포
supabase functions deploy append-workbook-row --no-verify-jwt
supabase functions deploy sign-application-workbook --no-verify-jwt
supabase functions deploy reset-application-workbook --no-verify-jwt
```

[object Object]

이 부록의 용도

"어떤 키를 어디에 두라고 했지?" 가 한 페이지에. 보안 사고는 80%가 '잘못된 위치에 둔 키' 에서 나옵니다.

[object Object]

키 이름	어디에 두나	누가 보나	사용처
VITE_SUPABASE_URL	<code>.env.local</code> (로컬) + GitHub Secret	누구나 (빌드 후 클라에 박힘)	프론트엔드
VITE_SUPABASE_ANON_KEY	<code>.env.local</code> + GitHub Secret	누구나	프론트엔드
VITE_BASE	<code>.github/workflows/pages.yml</code> 의 env	빌드 단계	Vite base 경로
SUPABASE_SERVICE_ROLE_KEY	Supabase secrets + GitHub Secret(백업용만)	⚠ 개 발자만	Edge Function · 백업 워크플로우
WORKBOOK_WEBHOOK_SECRET	Supabase secrets + Database Webhook 헤더	⚠ 개 발자만	append-workbook-row 인증
WORKBOOK_BUCKET	Supabase secrets	보통	함수들
WORKBOOK_OBJECT_KEY	Supabase secrets	보통	함수들
TEMPLATE_PUBLIC_URL	Supabase secrets	공개 OK	reset-application-workbook
WORKBOOK_RESET_ALLOWED_EMAILS	Supabase secrets	보통	sign.reset 함수
DB Password	본인 메모장 (오프라인)	본인만	DB 직접 접속 시
관리자(admin@admin.com) 비번	회사 비밀번호 금고	인수인계 대상	관리자 로그인

[object Object]

키	절대 X 위치	이유
SUPABASE_SERVICE_ROLE_KEY	코드, README, 메일, 메신저, VITE_* 환경 변수	DB 풀권한. 누설 즉시 모든 데이터 위험
WORKBOOK_WEBHOOK_SECRET	코드, README	웹훅 위장 가능
DB Password	GitHub, Slack, 메일	DB 직접 접근 가능
관리자 비밀번호	평문 메모, 공유 문서	어떤 백업·관리 작업도 가능

[object Object]

키	주기	절차
관리자 비번	분기 1회	Supabase Auth → Users → Reset password
service_role	6개월 1회·노출 의심 시 즉시	Project Settings → API → Reset → Supabase secrets · GitHub Secret 둘 다 갱신 → CI 재실행
WORKBOOK_WEBHOOK_SECRET	노출 의심 시	Supabase secrets 갱신 + Database Webhook 헤더 같은 값으로
anon key	회전 거의 X (필요 시 가능)	Reset 후 .env /Secret 갱신
DB Password	6개월 1회	Settings → Database → Reset password (직접 접속 도구 다 갱신)

[object Object]

```
# 어떤 시크릿이 등록되어 있는지
supabase secrets list

# Edge Function 목록
supabase functions list

# 로컬 환경변수가 잡히는지
echo $VITE_SUPABASE_URL      # zsh
```

⚠ Supabase secret 값 자체는 다시 못 봅니다. 메모장 따로 보관 필수.

[object Object]

이 부록의 용도

MVP 가 끝난 후 "있으면 좋다" 항목들. 한 항목씩 필요한 것만 골라 적용.

[object Object]

언제 좋나: 회사 메일 안내문에 GitHub URL 보다 자체 도메인이 보이는 게 신뢰감 ↑.

1. 도메인 1개 구입 (Namecheap/가비아 등) — 보통 만 원/년대.
2. GitHub 리포 → Settings → Pages → **Custom domain** 에 도메인 입력 → Save.
3. 도메인 관리 페이지에서 **CNAME** 레코드 추가
 - 호스트: **apply** → 값: **__GITHUB_ID__.github.io**
4. 1~30분 후 GitHub Pages 가 자동으로 HTTPS 발급.
5. **Enforce HTTPS** 체크 On.
6. **vite.config.js** 의 **base** 를 **'/'** 로 변경 (도메인 루트 사용 시) → push.

⚠ Supabase Auth 의 redirect URL, Edge Function CORS 가 새 도메인을 허용하는지 확인.

[object Object]

언제 좋나: 등록부 4칸만으로 부족, "문자 인증 1회"까지 받고 싶을 때.

1. Supabase Authentication → Phone provider 켜기 (Twilio/메시지 비용 발생).
2. AI에게:

Cursor에 그대로 복사

```
@App.jsx 7장 검증 통과 후 다음 단계로 가기 전에
사용자가 입력한 휴대폰 풀 번호로 supabase.auth.signInWithOtp({ phone })
호출 → 6자리 OTP 입력 → verifyOtp 통과해야 setStep('options').
실패 시 한국어 안내. 변경 후 Apply.
```

⚠ 비용·발신 한도 확인 필수. MVP 단계엔 **선택**.

[object Object]

언제 좋나: 신청이 들어오자마자 "1건 도착" 알림 받고 싶을 때.

1. Resend (<https://resend.com>) 가입 → API 키 발급 (무료 100건/일).
2. Supabase secret 등록:

```
supabase secrets set RESEND_API_KEY="re_xxx" \
  ALERT_EMAIL_TO="admin@admin.com" \
  ALERT_EMAIL_FROM="onboarding@resend.dev"
```

3. AI에게:

Cursor에 그대로 복사

```
supabase/functions/notify-new-application/index.ts 를 만들어줘.
동작:
- POST 로 webhook 호출 받기 (x-workbook-secret 인증 동일)
- body.record 에서 receipt_no, dong, ho, customer_name, total_amount 만 추출
- Resend API 로 ALERT_EMAIL_TO 에게 이메일 1통 발송
  from: ALERT_EMAIL_FROM
  subject: "[옵션신청] {접수번호} {동}-{호} {이름}"
  text: 간단한 본문 (개인정보 자세히 X - 접수번호와 평형 위주)
verify_jwt = false. config.toml에도 추가.
```

4. Database Webhook 1개 더 만들기: applications INSERT → 위 함수 URL.

⚠ 메일 본문에 주민번호·서명 등 민감정보 X. 접수번호·평형까지만.

[object Object]

언제 좋나: 시즌 끝난 한참 뒤 자료가 필요할 때를 대비.

1. GitHub에 사설 리포 1개 만들기 (`yyc-options-backups`).
2. 18-3 워크플로우에 단계 추가:

Cursor에 그대로 복사

```
.github/workflows/backup-workbook.yml 마지막 step 다음에
사설 리포 (yyc-options-backups) 의 main 브랜치로
backups/ 폴더 push 하는 step 을 추가해줘.
peaceiris/actions-gh-pages 같은 액션 대신 일반 git push 사용.
credentials 는 secrets.GH_BACKUP_TOKEN (Personal Access Token, repo scope) 로.
```

⚠ 사설 리포는 절대 Public 으로 바꾸지 말기. 개인정보 들어 있음.

[object Object]

본 매뉴얼은 "관리자 1명만" 기본. 만약 "여러 직원이 보는 화면" 이 필요하면:

1. Authentication → Email signups **여전히 OFF**.
2. Supabase Auth → Users → Invite 로 이메일 초대.

3. SQL:

```
INSERT INTO public.app_admins(email)
VALUES ('staff1@회사.com'), ('staff2@회사.com');
```

4. 직원도 `?admin=1` 로그인 후 화면 사용 가능.

절대 "Email signups ON + 화이트리스트 X" 조합으로 두지 말기. 외부인이 가입만 하면 RLS 밖에서 정책 회피 시도 가능.

[object Object]

`index.html` 의 CSP (17-6) 를 운영 단계에선 다음처럼 좁힐 수 있음.

```
<meta http-equiv="Content-Security-Policy" content="
  default-src 'none';
  base-uri 'self';
  frame-ancestors 'none';
  form-action 'self';
  script-src 'self';
  style-src 'self' 'unsafe-inline';
  img-src 'self' data:;
  font-src 'self' data:;
  connect-src 'self' https://__PROJECT_REF__.supabase.co;
" />
```

→ Supabase URL 한 도메인만 허용. 외부 스크립트는 100% 차단.

[object Object]

언제 좋나: "앱으로 깔아 놔야 안내하기 편함" 같은 상황.

가장 빠른 길: **PWA (홈 화면에 설치)**. 별도 스토어 등록 불필요.

Cursor에 그대로 복사

```
이 Vite 앱을 PWA 로 만들어줘. vite-plugin-pwa 사용.
manifest 의 name 은 "OO아파트 옵션 신청", short_name 은 "옵션신청".
아이콘은 public/pwa-192.png, public/pwa-512.png (자리만 잡고, 실제 이미지는 후속).
오프라인 시 "인터넷 연결을 확인해 주세요" 안내 페이지 1장.
변경 후 Apply.
```

`https://...` HTTPS 라면 Safari·Chrome 에서 "홈 화면에 추가" 로 설치됨.

[object Object]

- Supabase Pro 플랜 (\$25/월) 으로 업그레이드: PITR 백업·동시접속·DB 크기 ↑.
- Edge Function 의 무거운 작업(엑셀 read·write)을 가능한 짧게 (지금 코드 OK).
- 마감 1시간 동안의 폭증이 예상되면 **마감 시간 분산 안내** (예: 동·호 끝자리별 시간대).
- 모니터링: Supabase Logs + GitHub Actions backup 의 빈도/실패 알림 (이메일).

[object Object]

신청 1건마다 Slack 채널에 한 줄 알림.

Cursor에 그대로 복사

```
supabase/functions/notify-slack/index.ts 를 만들어줘.  
Webhook 으로 신청 INSERT 받기 (x-workbook-secret 인증).  
SLACK_WEBHOOK_URL 시크릿으로 Incoming Webhook 호출.  
메시지: "[음션신청] {receipt_no} 동{dong}-{ho} {customer_name} 함께 {total_amount}원"  
verify_jwt = false. config.toml 에도 추가.
```

같은 webhook 에 Database Webhook 1개 더 연결.

[object Object]

이 매뉴얼(docs/*.*.md) 도 운영 자산. 다음을 권장:

- 모든 챕터 파일을 PDF 로 1회 export → 사내 드라이브에 보관 (전기 끊겨도 종이로 출력 가능).
- Cursor에:

Cursor에 그대로 복사

```
docs/ 폴더의 모든 .md 를 합쳐 docs/MANUAL-FULL.md 한 장으로 묶고,  
목차 자동 생성. 변경 후 Apply.
```

→ 그 파일 1장으로 외부에 인쇄·공유.

[object Object]

여기까지 정말 수고하셨습니다.

0장에서 "Cursor 설치" 한 줄로 시작해서, 지금은:

- 사용자 사이트 (GitHub Pages)
- 인터넷 엑셀(DB·Storage)
- 자동 누적 엑셀 (Webhook + Edge Function)
- 관리자 운영 화면 (목록·상세·다운로드·초기화)
- 자동 배포 (GitHub Actions)
- 보안 잠금 (RLS·XSS·Signed URL)
- 운영 점검표·인수인계 문서·자동 백업

까지 한 세트가 본인 손으로 굴러갑니다.

매뉴얼은 한 번에 다 외울 필요 전혀 없어요.

막힐 때마다 "해당 장 + Cursor + 에러 메시지 한 줄" 이 모든 답입니다.

새 시즌 1번씩 돌릴 때마다 본 매뉴얼에 본인 메모(에러·해결·팁) 를 한 줄씩 더해 두세요.

1년 뒤 본인이 가장 든든해 합니다. 화이팅.